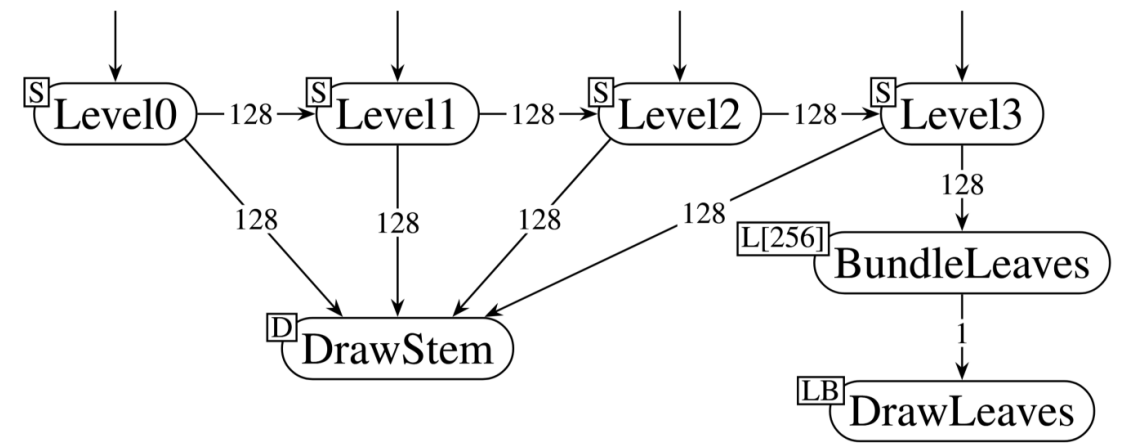
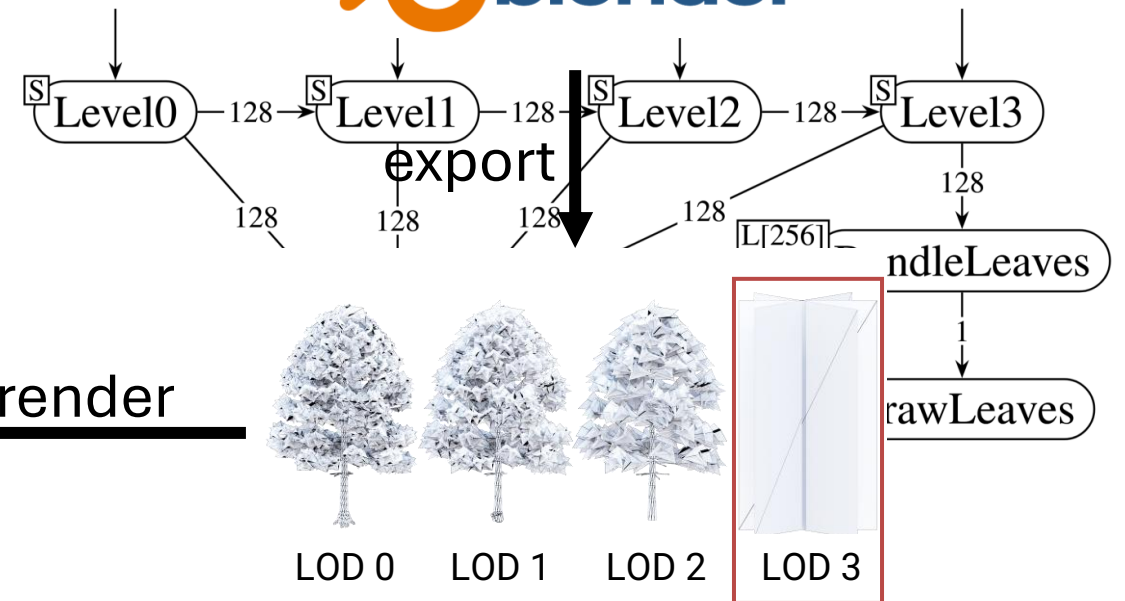






parameters



lots of geometry → high memory demand

compression?

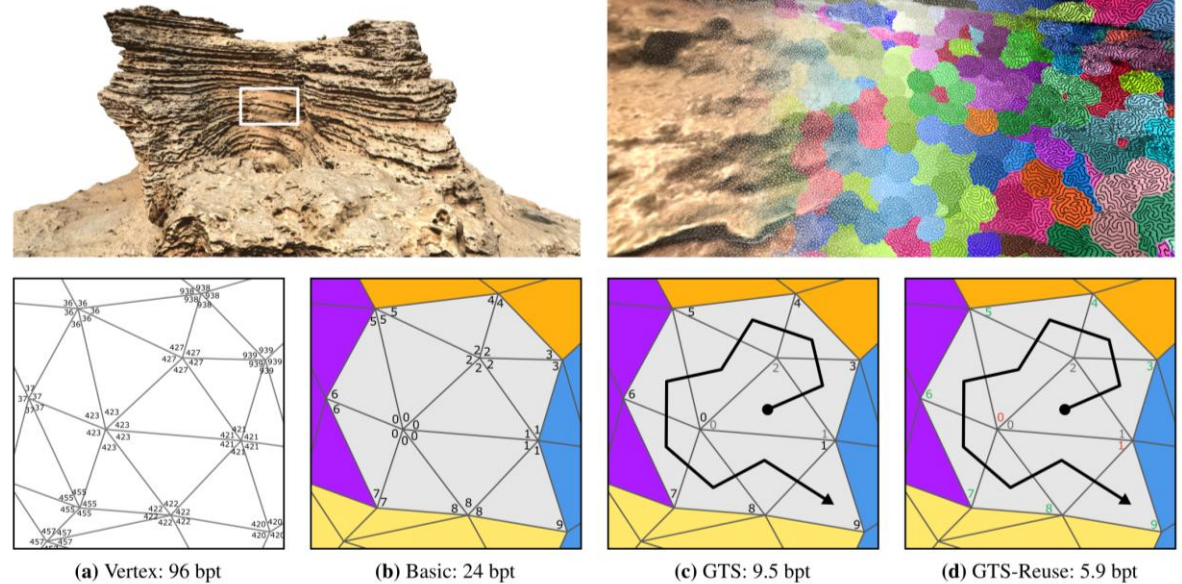


[PBRT] ca. 260 MiB

### Towards Practical Meshlet Compression

Bastian Kuth<sup>1</sup> Max Oberberger<sup>2</sup> Felix Kawala<sup>1,2</sup> Sander Reitter<sup>1,2</sup> Sebastian Michel<sup>1</sup> Matthäus Chajdas<sup>2</sup> Quirin Meyer<sup>1</sup>

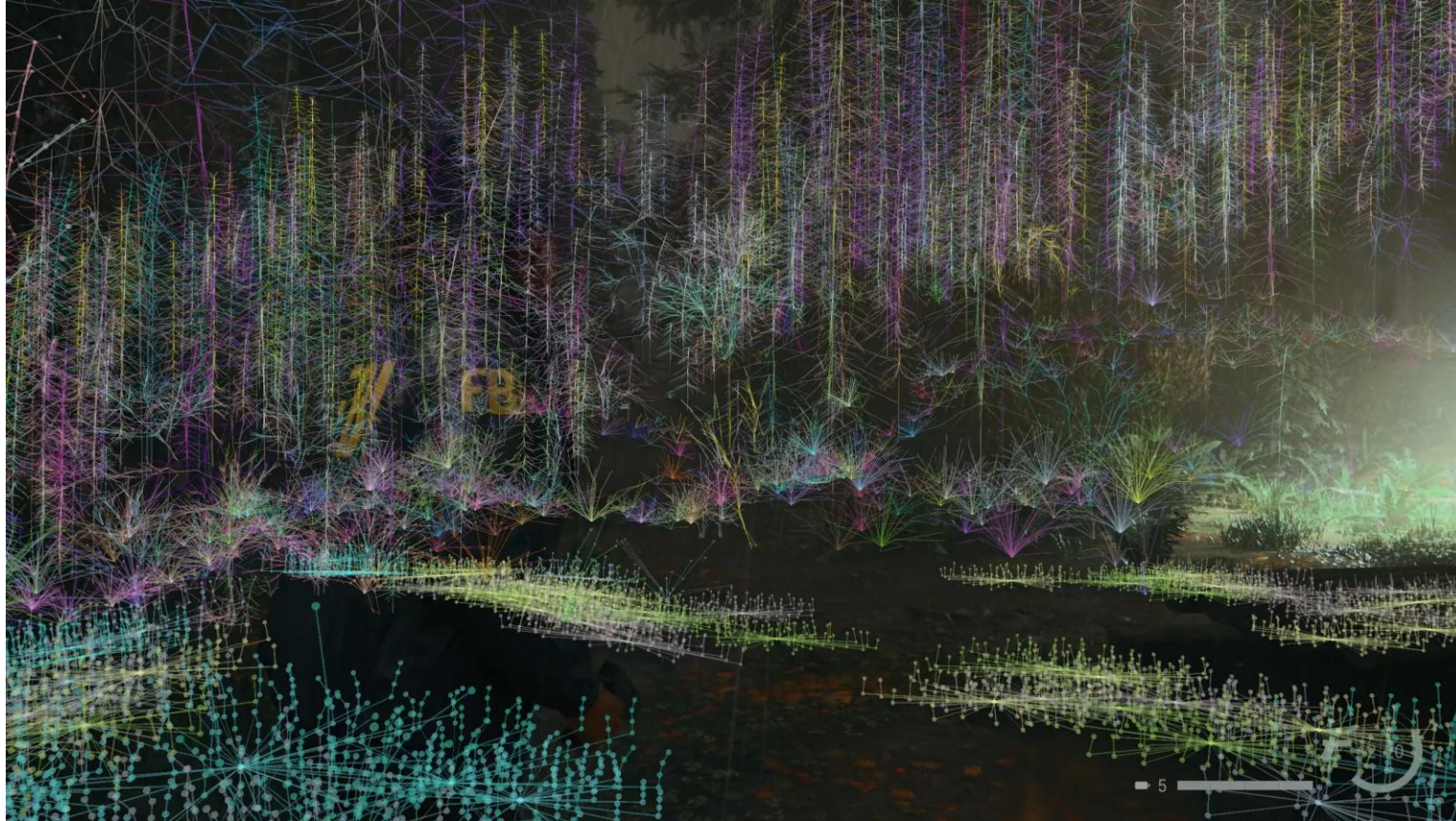
<sup>1</sup>Coburg University of Applied Sciences and Arts, Germany <sup>2</sup>AMD, Germany



→ only maybe 2.5x smaller

lots of geometry → high memory demand  
static geometry → animation?

rigging?



→ 300,000 bones!

lots of geometry → high memory demand

static geometry → animation?

discrete LOD → popping

tessellation?



→ limited to higher LODs

lots of geometry → high memory demand  
static geometry → animation?  
discrete LOD → popping  
slow generation → lowers artist productivity

## Real-Time Procedural Generation with GPU Work Graphs

BASTIAN KUTH, Coburg University, Germany

MAX OBERBERGER, AMD, Germany

CARSTEN FABER, Coburg University, Germany

DOMINIK BAUMEISTER, AMD, Germany

MATTHÄUS CHAJDAS, AMD, Germany

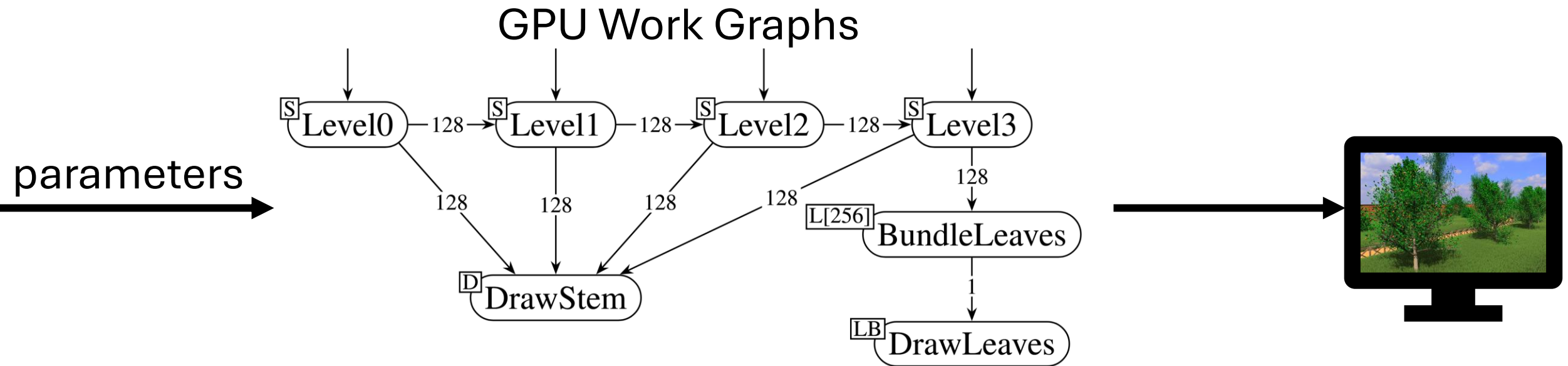
QUIRIN MEYER, Coburg University, Germany



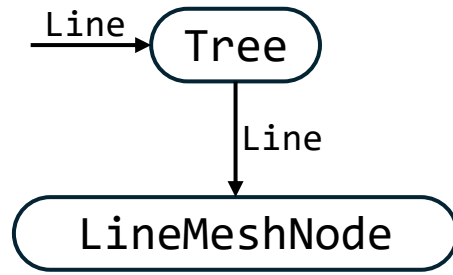
(a) Without our system

(b) With our system

lots of geometry → high memory demand  
static geometry → animation?  
discrete LOD → popping  
slow generation → lowers artist productivity



## GPU Work Graphs



```
struct Line {float2 a, b};
```

```
[Shader("node")]  
[NodeLaunch("thread")]
```

```
void Tree(  
    Line line  
) {
```

```
}
```

```
}
```

```
[Shader("node")]  
[NodeLaunch("mesh")]  
[NodeId("LineMeshNode", 0)]
```

```
void MeshShader(  
    Line line  
) {
```

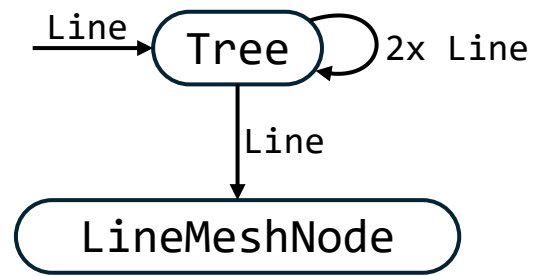
```
{
```

```
}
```

```
float4 PixelShader() : SV_TARGET {
```

```
}
```

## GPU Work Graphs

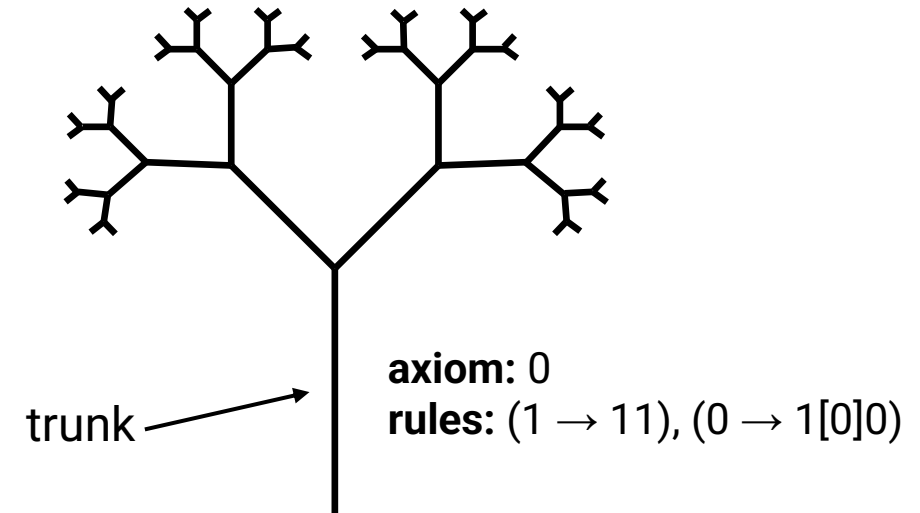


```
struct Line {float2 a, b};
```

```
[Shader("node")]
[NodeLaunch("thread")]

void Tree(
    ThreadNodeInputRecord<Line> ir,
    [MaxRecords(1)] NodeOutput<Line> LineMeshNode,
) {
    ThreadNodeOutputRecords<Line> d =
        LineMeshNode.GetThreadNodeOutputRecords(1);
    d.Get(0) = ir.Get(); d.OutputComplete();
}
```

```
[Shader("node")]
[NodeLaunch("mesh")]
[NodeId("LineMeshNode", 0)]
[NodeDispatchGrid(1,1,1)][NumThreads(1,1,1)]
[OutputTopology("line")]
void MeshShader(
    DispatchNodeInputRecord<Line> ir,
    out indices uint2 lines[1],
    out vertices float4 verts[2] : SV_POSITION)
{
    SetMeshOutputCounts(2, 1);
    lines[0] = uint2(0,1);
    verts[0] = float4(ir.Get().a, 0, 1);
    verts[1] = float4(ir.Get().b, 0, 1);
}
float4 PixelShader() : SV_TARGET {
    return float4(0,0,0,1);
}
```





Creation and Rendering of Realistic Trees

Jason Weber<sup>1</sup> Joseph Penn<sup>2</sup>  
Teletronics International, Inc. Army Research Laboratory

"From such small beginnings - a mere grain of dust,  
as it were - do mighty trees take their rise."  
Henry David Thoreau from "Faith in a Seed"

ABSTRACT

Recent advances in computer graphics have produced images approaching the elusive goal of photorealism. Since many natural objects are so complex and detailed, they are often not rendered with convincing fidelity due to the difficulties in succinctly defining and efficiently rendering their geometry. With the increased demand of future simulation and virtual reality applications, the production of realistic natural-looking background objects will become increasingly more important.

We present a model to create and render trees. Our emphasis is on the overall geometrical structure of the tree and not a strict adherence to botanical principles. Since the model must be utilized by general users, it does not require any knowledge beyond the principles of basic geometry. We also explain a method to seamlessly degrade the tree geometry at long ranges to optimize the drawing of large quantities of trees in forested areas.

1 INTRODUCTION

Historically, much of the effort in computer graphics has been directed toward rendering precisely defined geometrical shapes such as manufactured objects whose geometry must be clear-cut and well-defined. CAD tools that are often used to design these objects can also be used to specify the geometrical properties in terms of simpler surfaces or solid geometric primitives. The complexity of many objects is generally low enough to allow complex lighting and ray-tracing computations that approach photorealism.

Natural objects offer a more profound challenge. A hillside may contain hundreds of trees, billions of grass blades, and countless rocks, pebbles, and ground variations. Each tree may easily be characterized by hundreds of thousands of leaves and thousands of branches, branchlets, and stems oriented in complex directions. A complex landscape could require an unimaginable large number of polygons to define every minute facet. As a result, complex natural backgrounds containing vegetation are often neglected in high quality image generation and scene simulation because of the difficulty of properly defining and rendering them in a reasonable time. Emphasis is placed on the buildings, vehicles, and assorted manufactured objects that are often the focus of the dominant action in a scene. Because of speed requirements, two-dimensional texture-mapped trees drawn as rotating billboards are common today in many real-time applications, but their appearance can be objectionable. This is especially evident when a viewer is in motion. See [ROHL94] for examples of 2D trees. As simulations become more realistic, the deficiencies in the background objects become more apparent.

We present a model to create and render trees. In designing this model, we have set guidelines focused on the requirements of scene simulation. The foremost requirement is the appropriate level of resolution and quality. For items to appear realistic in a dynamic simulation, the viewer must get the proper sense of rotational as well as translational motion when passing or circling objects.

Realism also depends on the accuracy of textural effects due to leaves and branches within the tree shadowing each other at various times of the day. Therefore, all trees must be three-dimensional. Fortunately, as background objects, trees would rarely be taller than 5 to 20 percent of screen height. Therefore, fine details such as leaf curvature and vein structure are not important. But, a tree's branch structure must be very accurate at this resolution. Leaves do not completely conceal the underlying branches of dormant or sparsely foliated trees.

<sup>1</sup> weber@teleport.com, now employed at Dynamics Research Corporation  
<sup>2</sup> joseph@belvoir-arl-irsig.army.mil

Permission to make digital hard copy of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication and its date appear, and notice is given that copying is by permission of ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.  
©1995 ACM 0-89791-701-4/95/0008...\$3.50

The model must be capable of creating a wide variety of actual tree types and related vegetation such as shrubs, bushes, and palms, as well as cacti and even large grasses. Shrubs, for example, can be easily simulated with the model since they really only differ from trees in that they are usually shorter and have multiple trunks originating directly from the ground [REIL91]. The model must be able to handle random parameters so that a very large number of structural variations can be generated from the design specifications of a particular tree species. It should also implement time-dependent oscillations due to wind and other perturbations.

Use of the model must be understandable by a common user with only a general knowledge of basic geometry, such as directly observable angles and lengths. This excludes the use of any model parameters requiring understanding of difficult principles such as differential equations. Likewise, the model must be stable and easy to use. User-entered free-form equations could easily cause unpredictable behavior. Aspects of the model that may be difficult to control should be isolated from the user and be represented by intuitive parameters. However, the model should not be constrained in a way that interferes with the user's freedom of design.

The specification for the tree must be compact and be able to recreate and render the tree geometry efficiently. This includes the ability to degrade geometry to low resolution at long ranges, where increased speed is necessary to render large forested areas. Any degradation must use negligible overhead and be seamless, even in dynamic simulations where ranges to trees are continuously changing.

This model was designed to successfully meet these criteria. We demonstrate the tree model in our natural environment scene generator. We have developed a compact but varied library of specification files for generating trees that are used in simulating a wide variety of landscapes.

The following section briefly discusses other tree models. The third section gives an overview of our observations of trees. The fourth section goes into the specific details and equations explaining how our parameters are used to create the geometric description. The fifth section explains our method of drawing optimally-degraded instances of the trees at longer ranges. The sixth section is a very short description of our project and how we use the trees in our application. An appendix includes a listing of our parameters and four sample tree specifications.

2 PREVIOUS MODELS

We will make some comparisons and contrasts to other tree models here and throughout the paper. We cannot fully explain the previous work in this space and will direct the reader to definitive references.

Honda introduced a model using parameters to define the skeleton of a tree [HOND71]. He clearly illustrated the difference between the monopodial and dichotomous branching. In dichotomous branching, the branches tend to split apart in different directions from the original. Monopodial branching tends to act similarly except that one branch continues in-line from the original. Honda assumes that monopodial branching is a special case concerning structures that are parallel to the line of gravity.

Lindenmayer introduced a string rewriting system [LIND68] for cellular interaction commonly called the L-system. This is later applied to plants and trees and is extensively described in his book with Prusinkiewicz [PRUS90] which describes the system with a few extensions, such as allowing for context-sensitivity and random variations. Basically, the string starts with a seed of a single character. A set of rules defines how to substitute characters during an iteration of rewriting. Presumably, any one character may be converted into several characters. This process is continued iteratively and the string grows. After a designated number of iterations, these strings can be interpreted as geometric commands. Rules can be selected to produce the monopodial or dichotomous branching, as desired.

Aono and Kuni stated that the L-system was not capable of producing complex three-dimensional patterns of branching [AONO84]. They demonstrated their models which also introduced interesting features such as attraction, inhibition, and statistical variations of angles. They made a



Plate 1: (a,b) Black Oak, (c,d) Sassafras, (e,f) Swamp Oak, (g) Cottonwood, (h) Tamarack, (i,j) Lombardy Poplar, (k) Queen Palm, (l) cactus, (m,n) Rock Elm, (o) Black Spruce, (p) Austrian Pine, (q,r) Quaking Aspen, (s) Balsam Fir, (t) White Cedar, (u) Bamboo, (v) generic, (w) Jack Pine, (x) Longleaf Pine

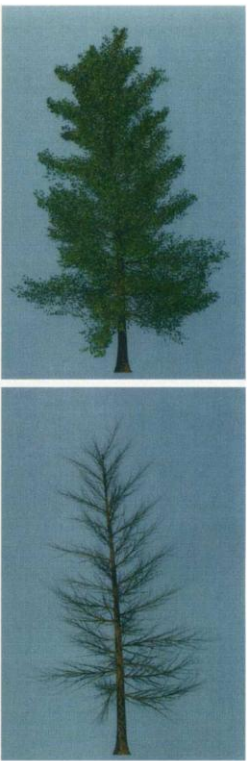


Plate 2: Black Tupelo, (a) with leaves (b) without leaves

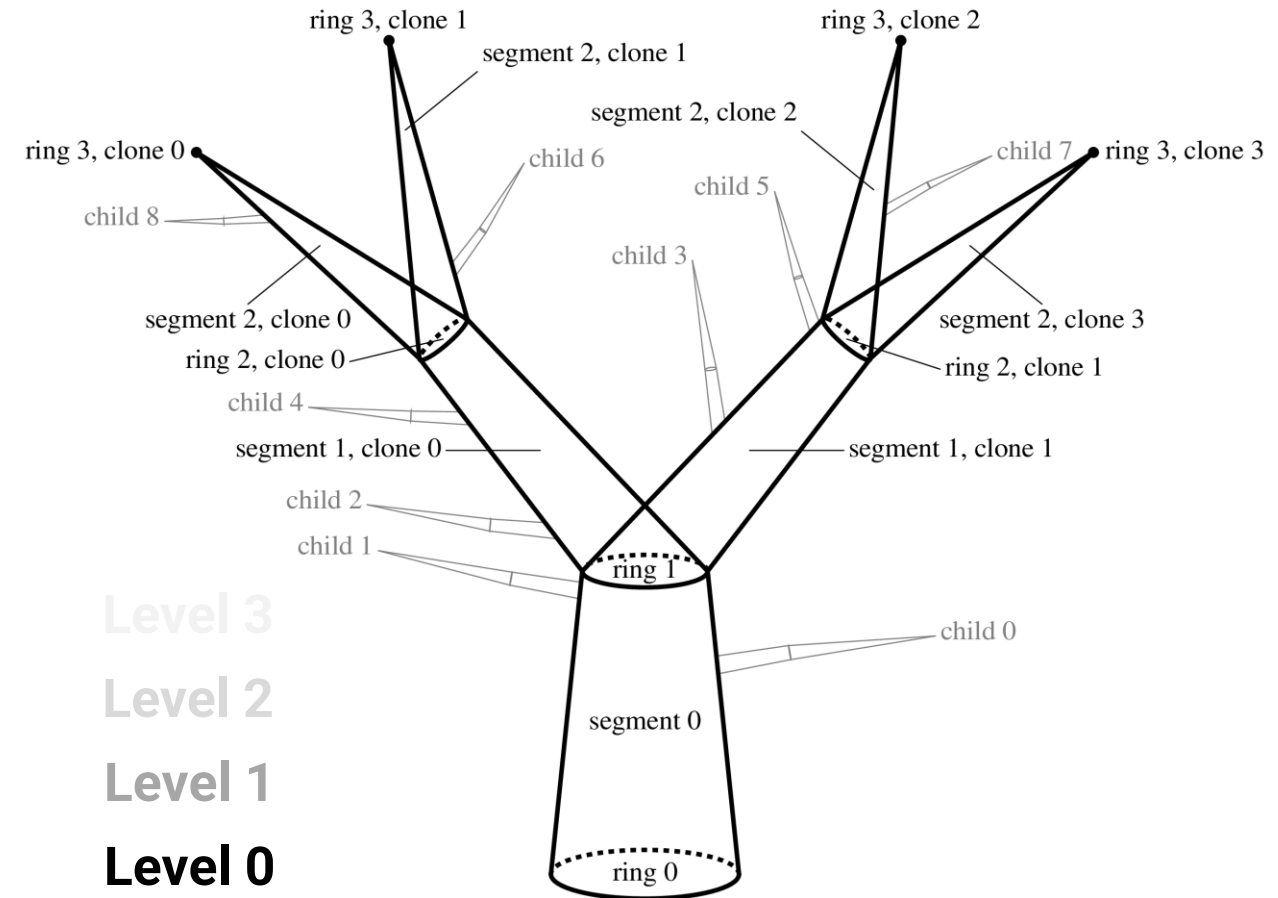
# Weber-Penn Model



## Creation and Rendering of Realistic Trees

Jason Weber<sup>1</sup>      Joseph Penn<sup>2</sup>  
Teletronics International, Inc.      Army Research Laboratory

| Type         | nLevels | nCurveRes0 | nChildren0 | ... |
|--------------|---------|------------|------------|-----|
| Sassafras    | 4       | 16         | 15         | ... |
| Black Tupelo | 4       | 10         | 50         | ... |
| Cherry       | 3       | 8          | 40         | ... |
| ⋮            | ⋮       | ⋮          | ⋮          | ⋮   |

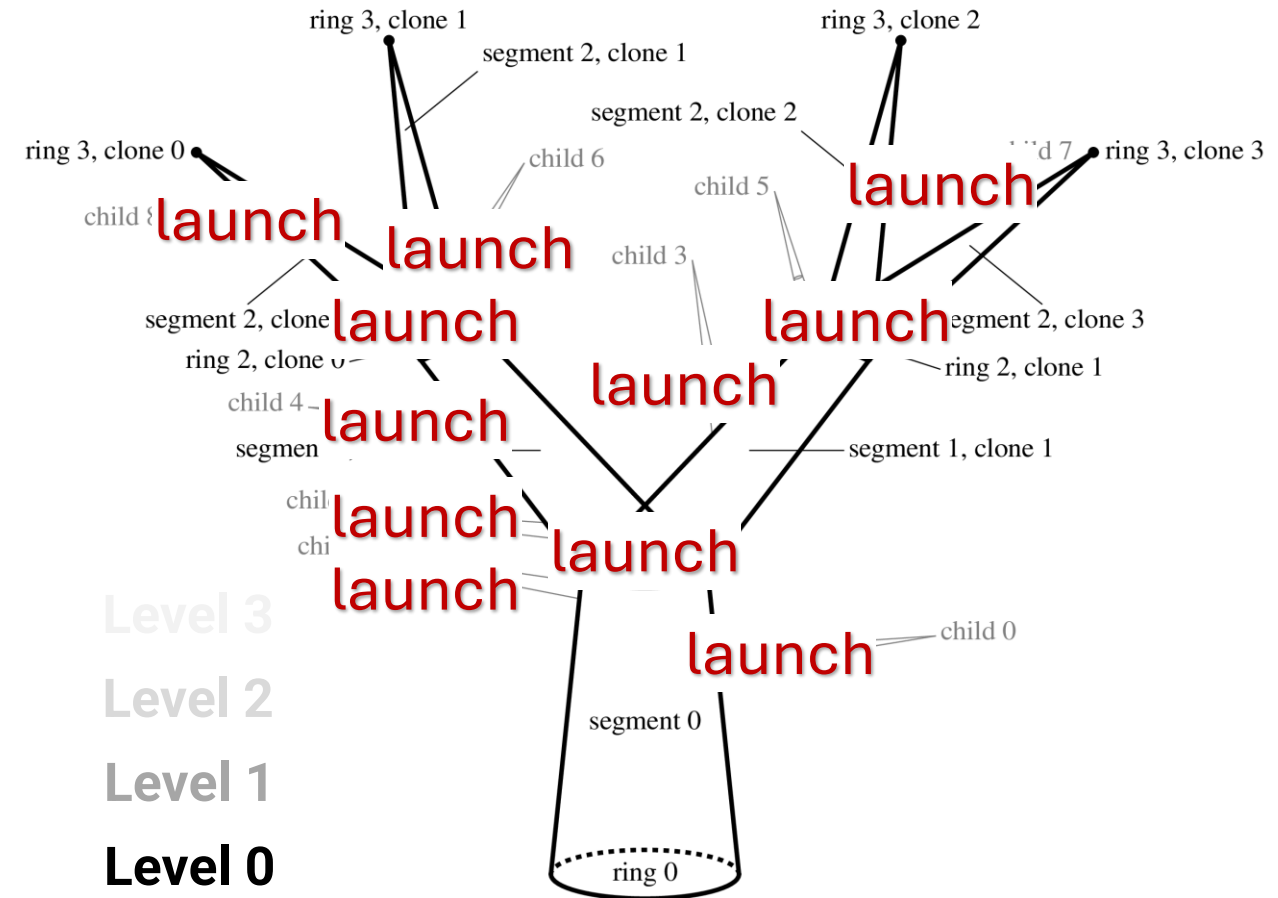


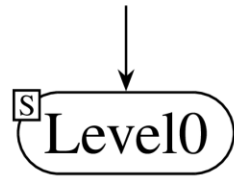
Implementation?

## Many Launches?

→ high launch overhead

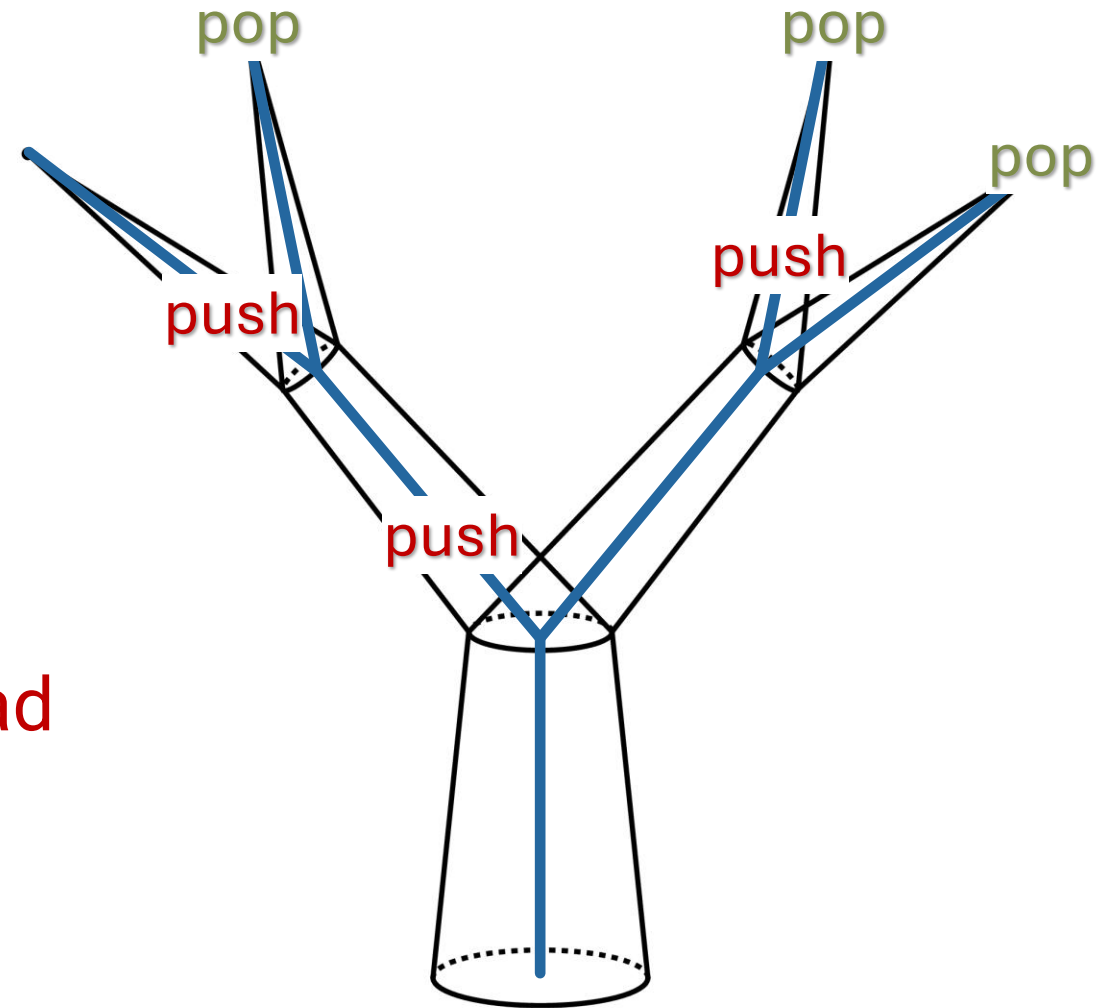
→ exceeds WG 32 depth limit





## Use Stack?

- complex memory management
- high thread divergence
- exceeds WG output limit per thread

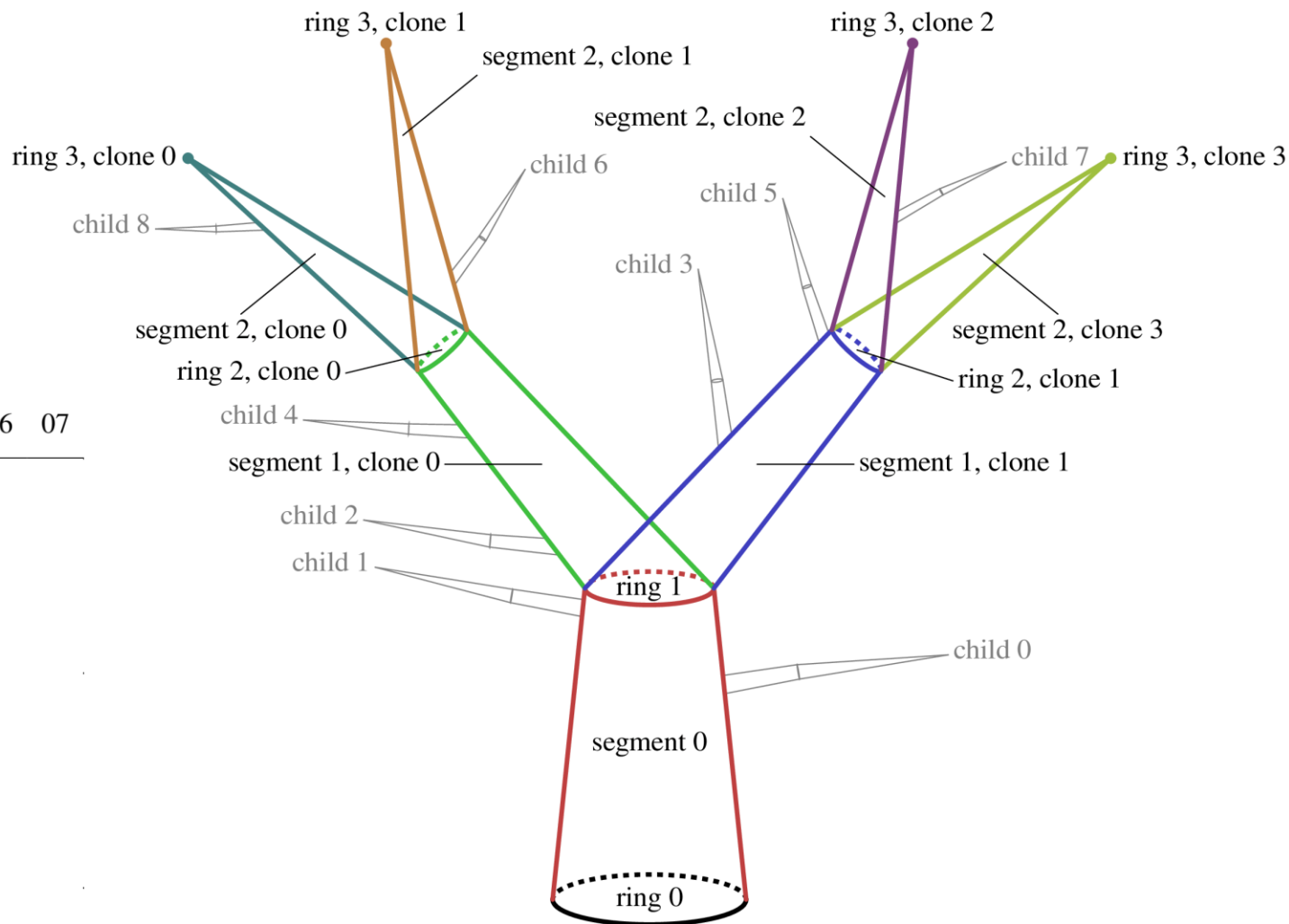


```

[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)

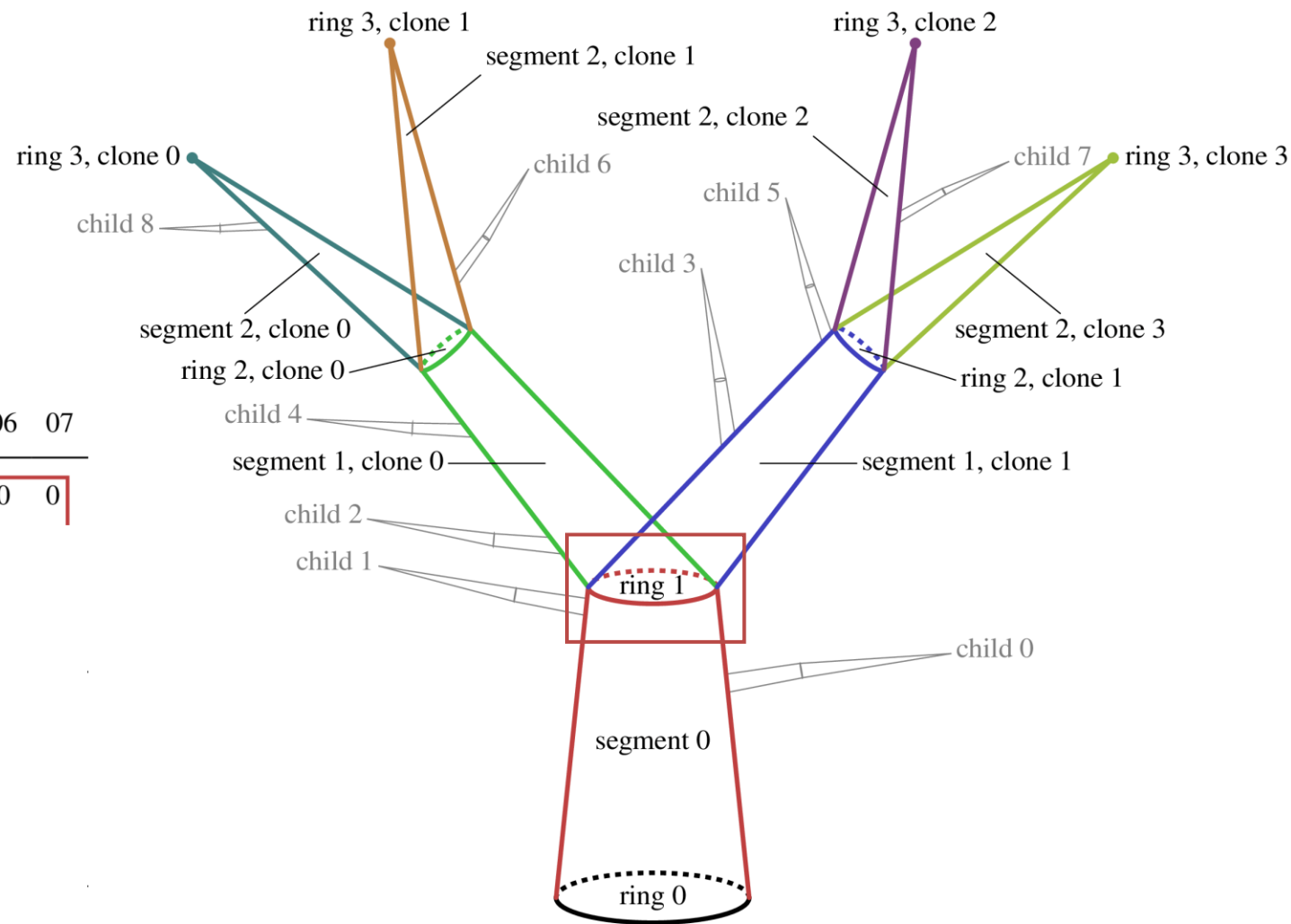
```

| thread | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|--------|----|----|----|----|----|----|----|----|
|        |    |    |    |    |    |    |    |    |



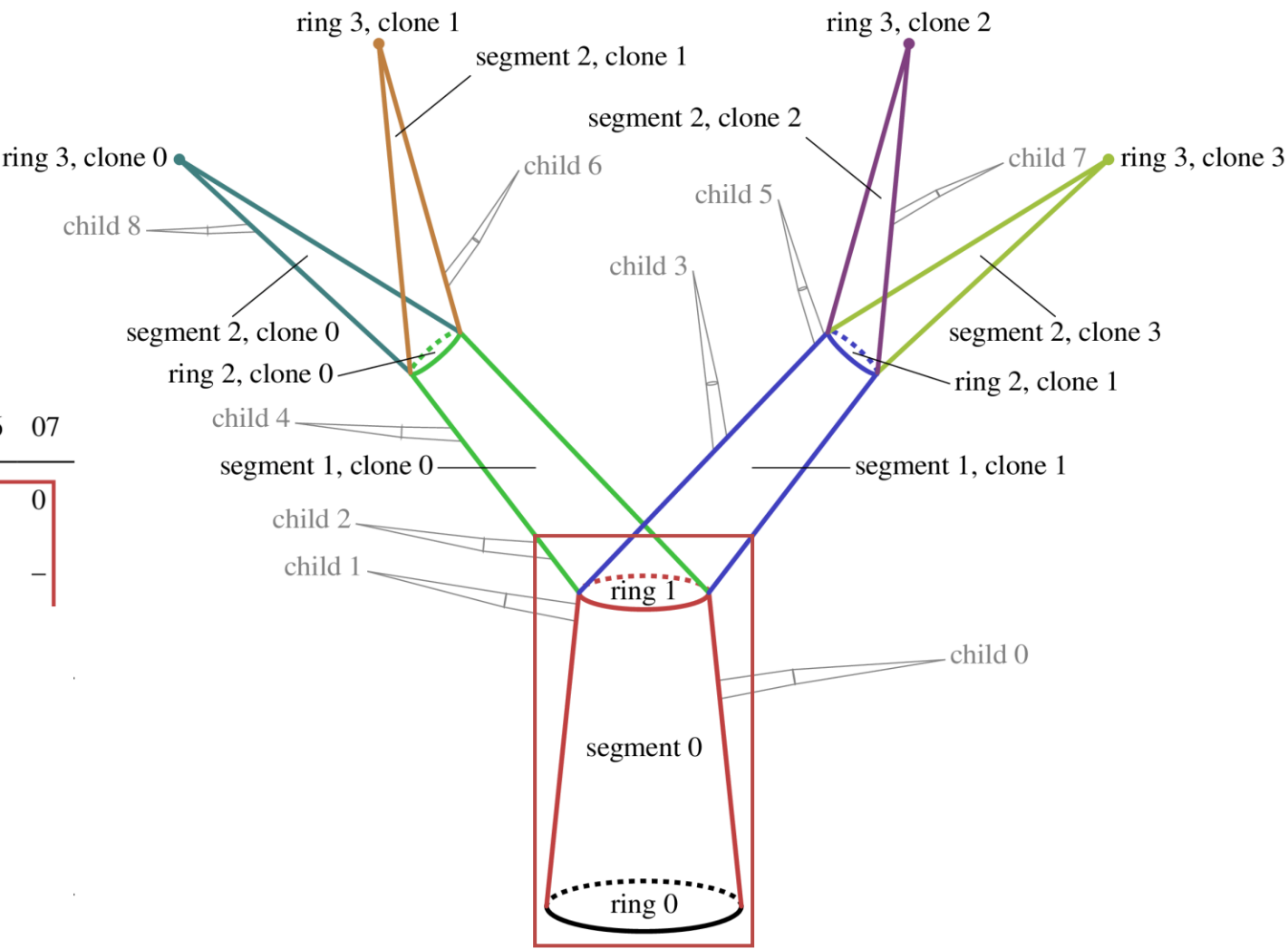
```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)
```

|  | thread                   | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|--|--------------------------|----|----|----|----|----|----|----|----|
|  | compute ring 1, clone... | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |



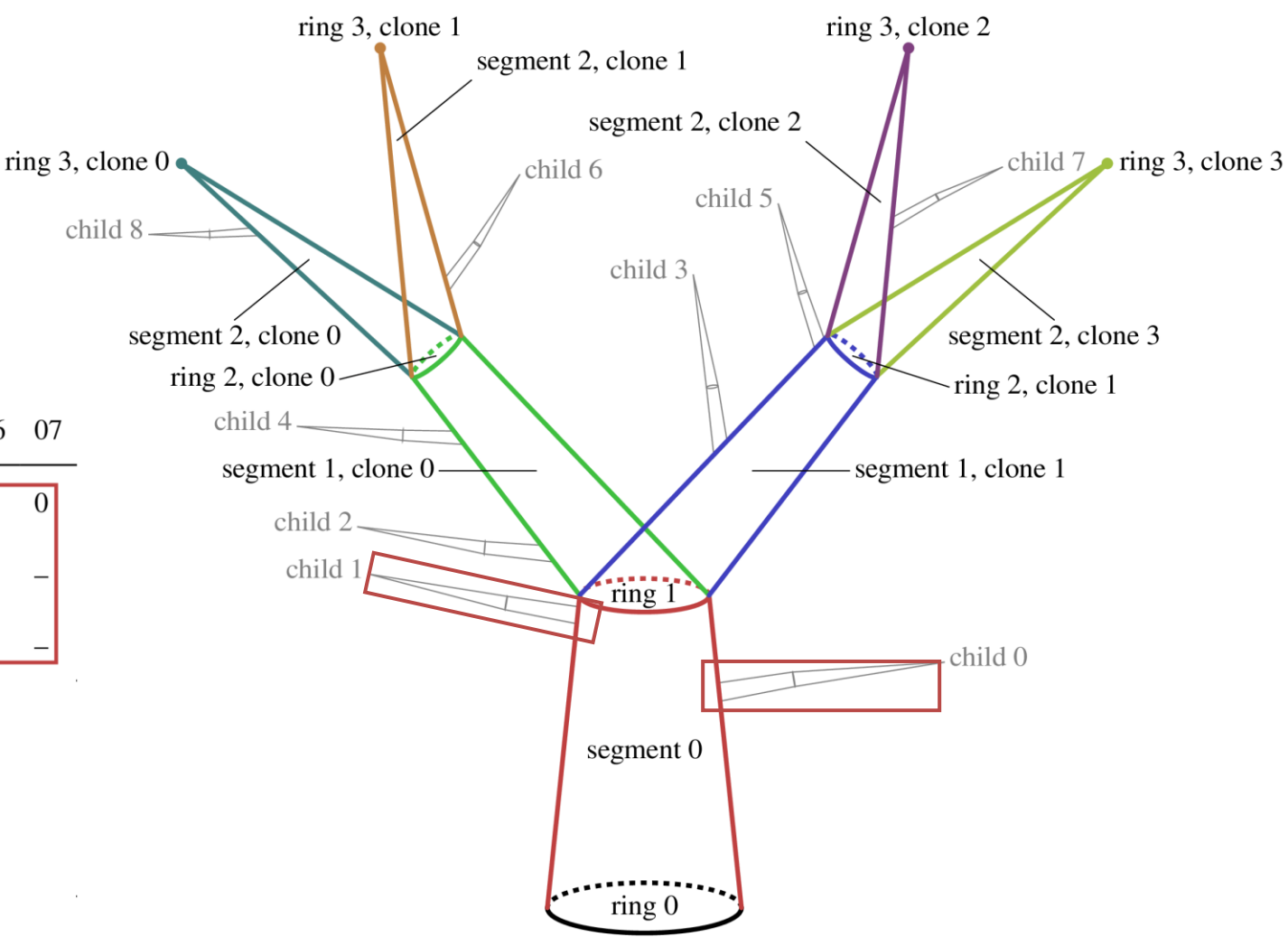
```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)
```

|             |                                                                                               | thread | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|-------------|-----------------------------------------------------------------------------------------------|--------|----|----|----|----|----|----|----|----|
| Iteration 0 | compute ring 1, clone...                                                                      |        | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">D</span> for segment 0, clone... |        | 0  | -  | -  | -  | -  | -  | -  | -  |



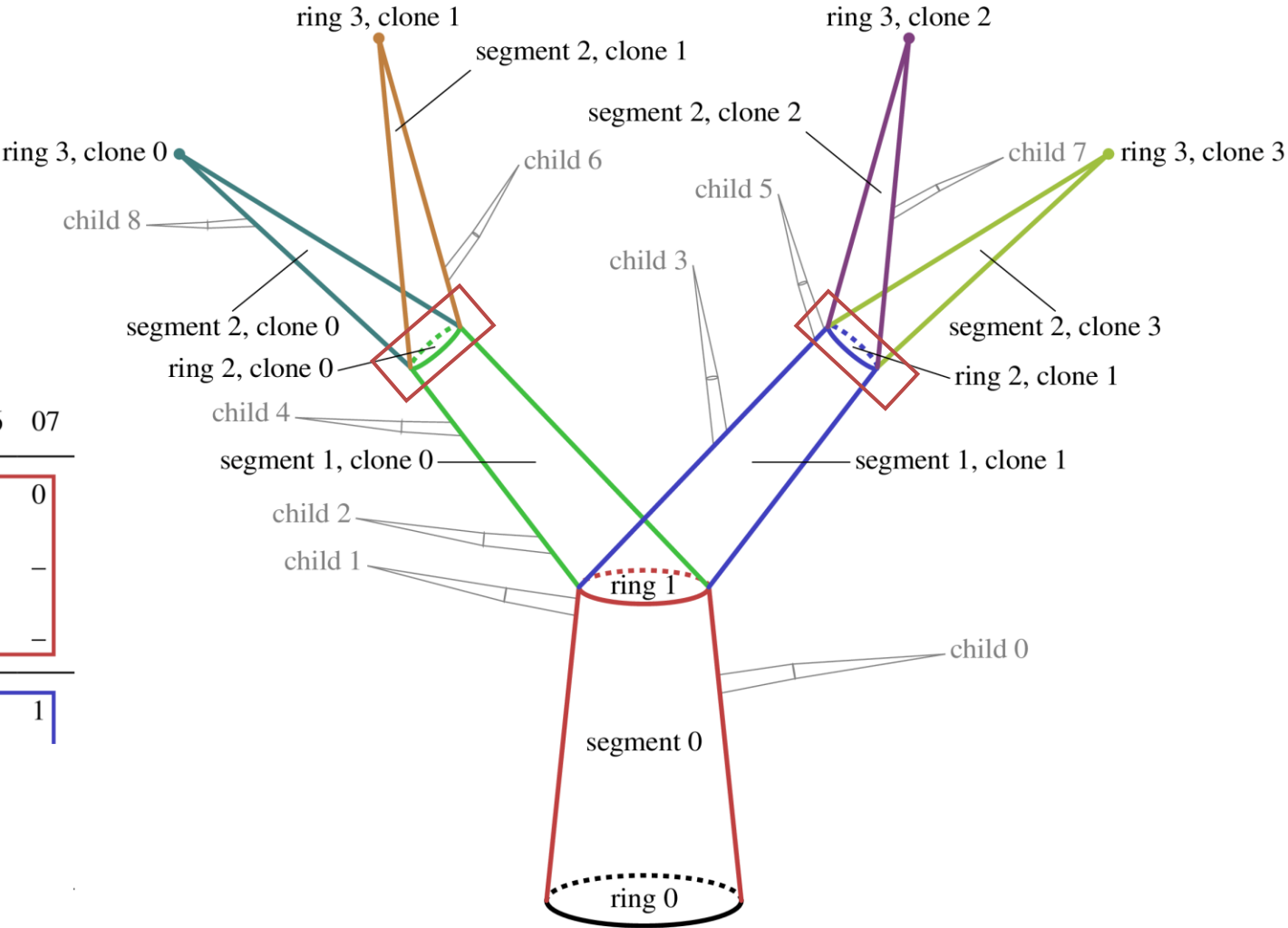
```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)
```

|             |                                                                                               | thread | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|-------------|-----------------------------------------------------------------------------------------------|--------|----|----|----|----|----|----|----|----|
| Iteration 0 | compute ring 1, clone...                                                                      |        | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">D</span> for segment 0, clone... |        | 0  | -  | -  | -  | -  | -  | -  | -  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">S</span> for child...            |        | 0  | 1  | -  | -  | -  | -  | -  | -  |



```
[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)
```

|             | thread                                                                                        | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|-------------|-----------------------------------------------------------------------------------------------|----|----|----|----|----|----|----|----|
| Iteration 0 | compute ring 1, clone...                                                                      | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">D</span> for segment 0, clone... | 0  | -  | -  | -  | -  | -  | -  | -  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">S</span> for child...            | 0  | 1  | -  | -  | -  | -  | -  | -  |
|             | compute ring 2, clone...                                                                      | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 1  |

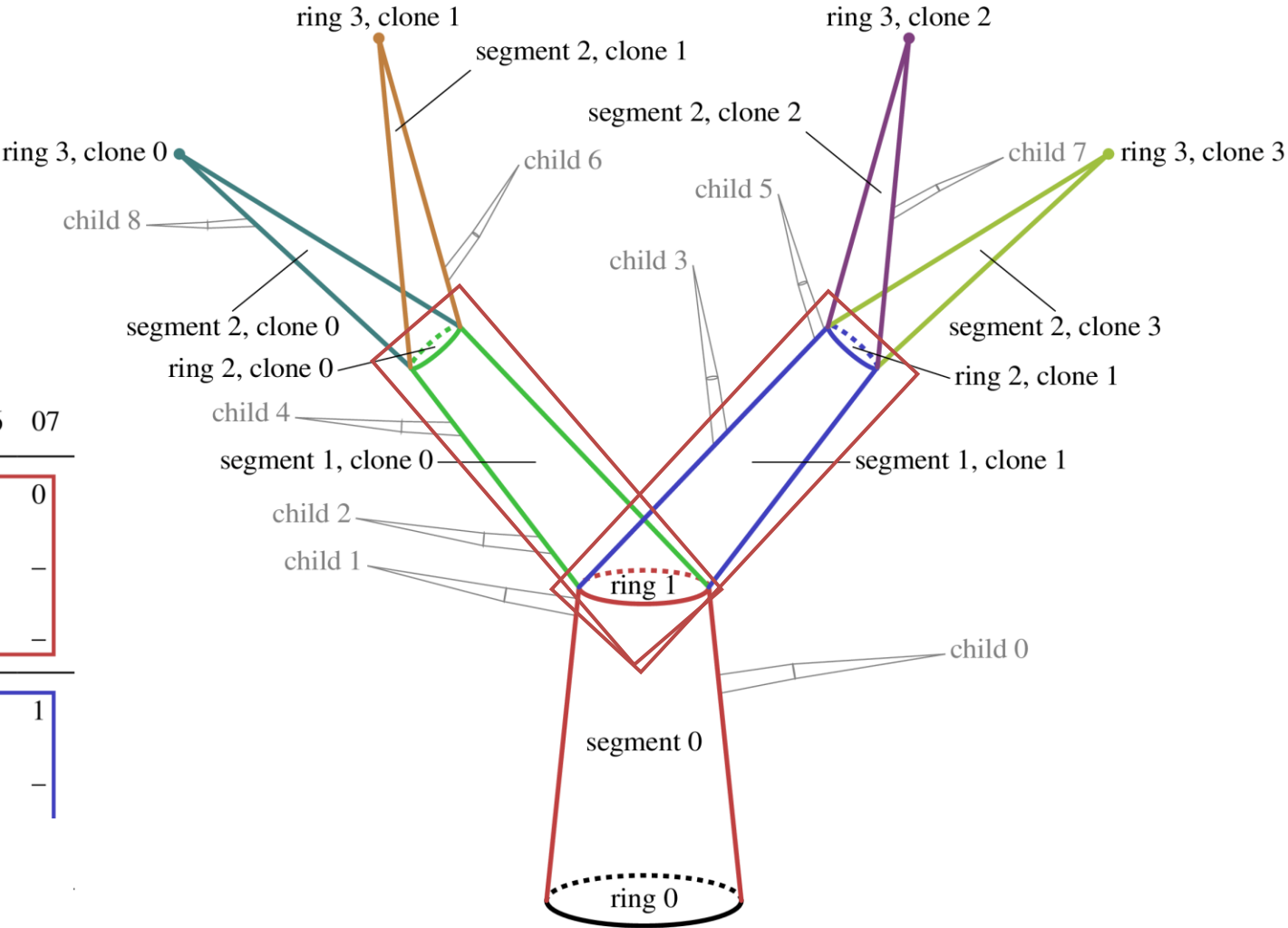


```

[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)

```

|             |                                                                                               | thread | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|-------------|-----------------------------------------------------------------------------------------------|--------|----|----|----|----|----|----|----|----|
| Iteration 0 | compute ring 1, clone...                                                                      |        | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">D</span> for segment 0, clone... |        | 0  | -  | -  | -  | -  | -  | -  | -  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">S</span> for child...            |        | 0  | 1  | -  | -  | -  | -  | -  | -  |
| Iteration 1 | compute ring 2, clone...                                                                      |        | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 1  |
|             | write <span style="border: 1px solid black; padding: 0 2px;">D</span> for segment 1, clone... |        | 0  | -  | -  | -  | 1  | -  | -  | -  |

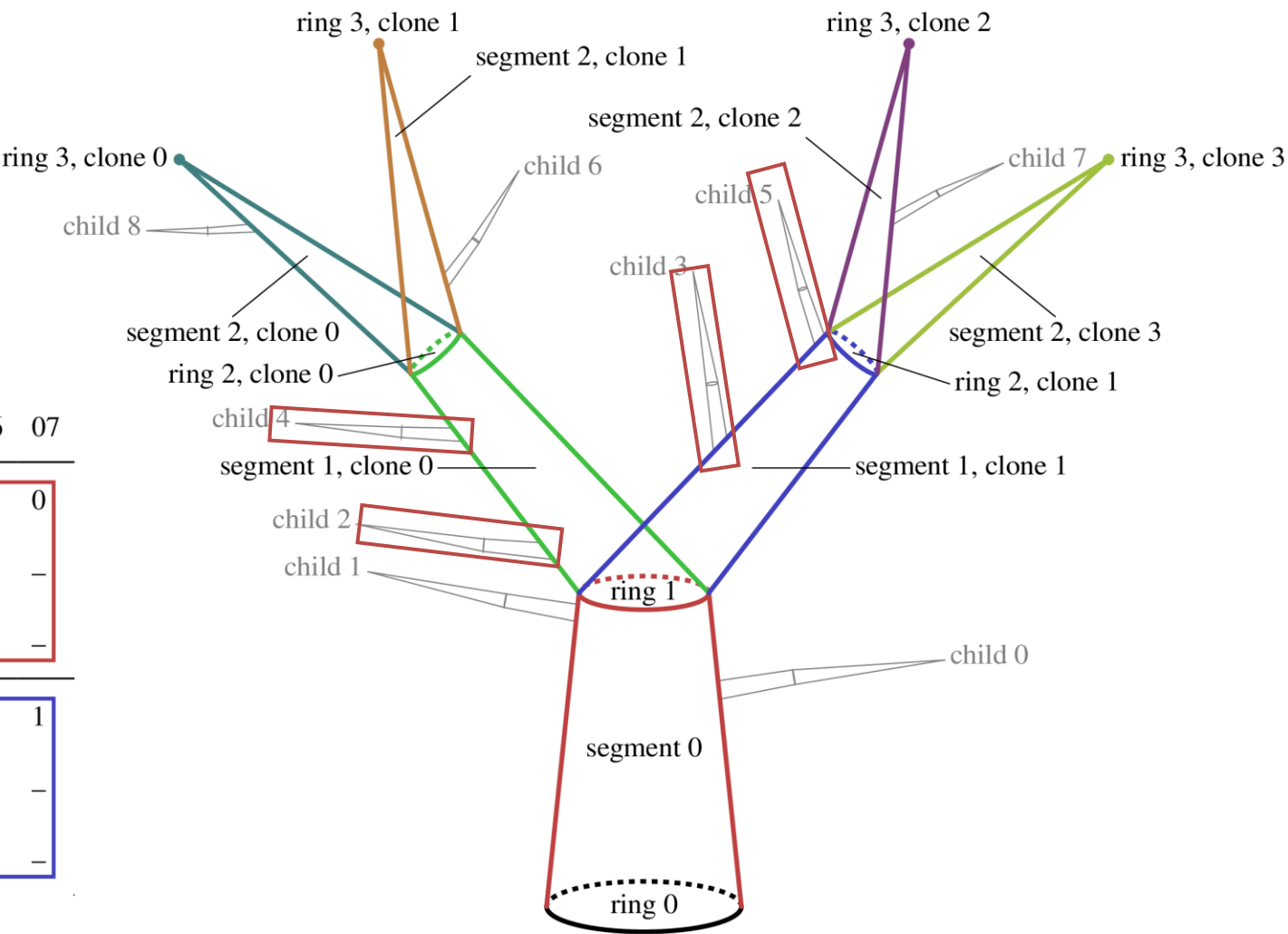


```

[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)

```

|             | thread                                       | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|-------------|----------------------------------------------|----|----|----|----|----|----|----|----|
| Iteration 0 | compute ring 1, clone...                     | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             | write <span>D</span> for segment 0, clone... | 0  | -  | -  | -  | -  | -  | -  | -  |
|             | write <span>S</span> for child...            | 0  | 1  | -  | -  | -  | -  | -  | -  |
| Iteration 1 | compute ring 2, clone...                     | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 1  |
|             | write <span>D</span> for segment 1, clone... | 0  | -  | -  | -  | 1  | -  | -  | -  |
|             | write <span>S</span> for child...            | 2  | 4  | -  | -  | 3  | 5  | -  | -  |

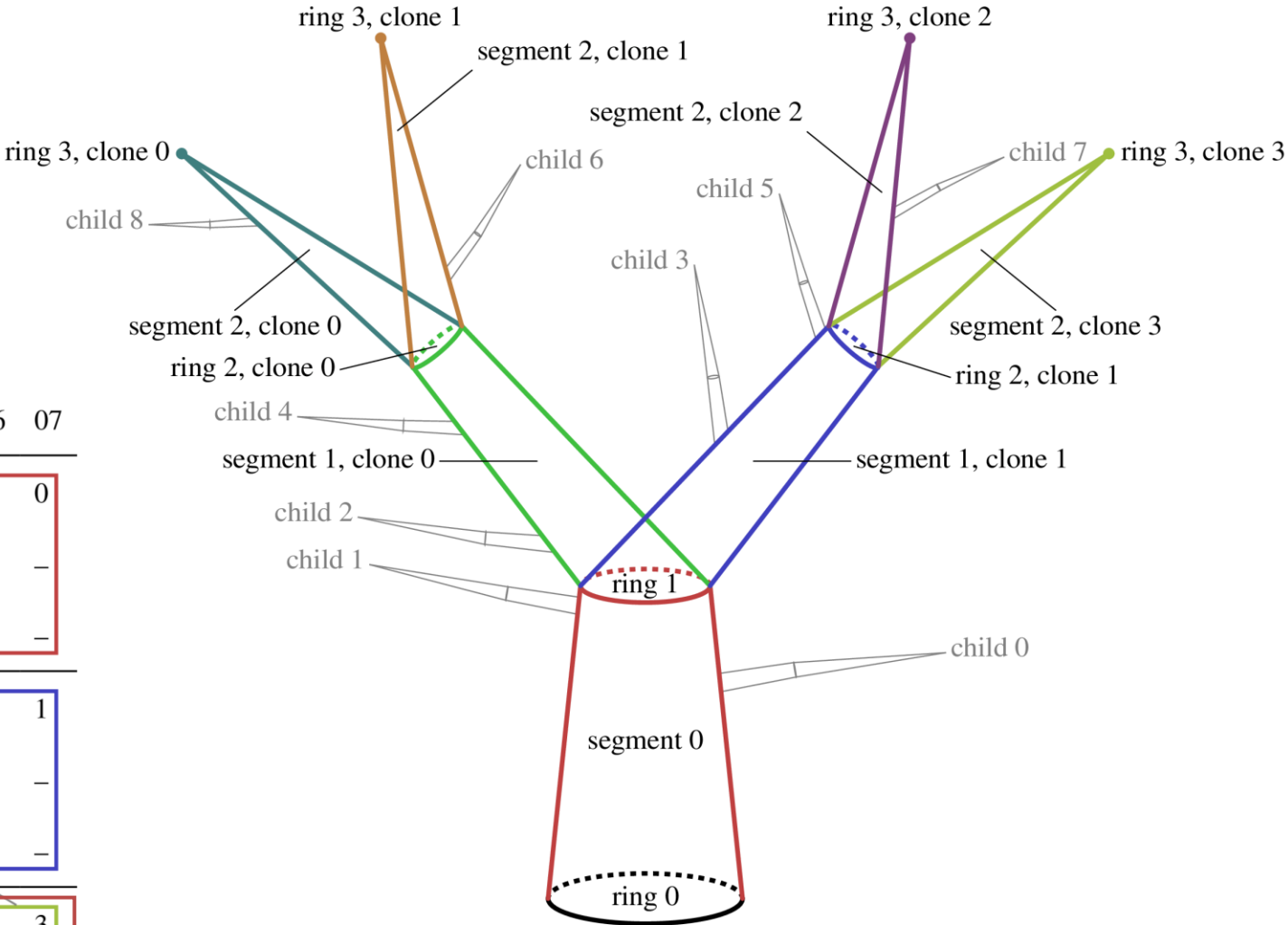


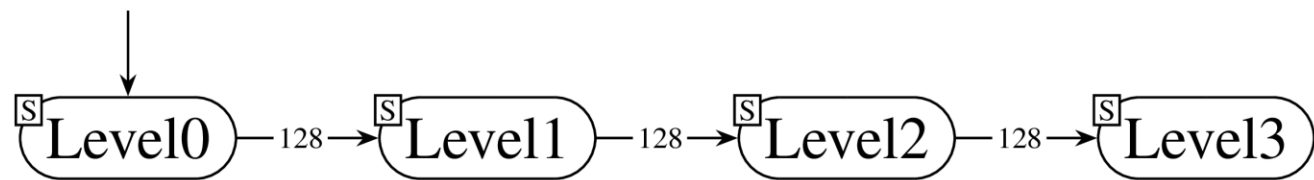
```

[Shader("node")]
[NodeLaunch("broadcasting")]
[NodeDispatchGrid(1, 1, 1)]
[NumThreads(32, 1, 1)]
void Level0(
    uint gtid : SV_GroupThreadId,
    DispatchNodeInputRecord<S> ir,
    [MaxRecords(128)] NodeOutput<S> nextLevel,
    [MaxRecords(128)] NodeOutput<D> drawStem,
    ...
)

```

|             |                                              | thread | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|-------------|----------------------------------------------|--------|----|----|----|----|----|----|----|----|
| Iteration 0 | compute ring 1, clone...                     |        | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
|             | write <span>D</span> for segment 0, clone... |        | 0  | -  | -  | -  | -  | -  | -  | -  |
|             | write <span>S</span> for child...            |        | 0  | 1  | -  | -  | -  | -  | -  | -  |
| Iteration 1 | compute ring 2, clone...                     |        | 0  | 0  | 0  | 0  | 1  | 1  | 1  | 1  |
|             | write <span>D</span> for segment 1, clone... |        | 0  | -  | -  | -  | 1  | -  | -  | -  |
|             | write <span>S</span> for child...            |        | 2  | 4  | -  | -  | 3  | 5  | -  | -  |
| Iteration 2 | compute ring 3, clone...                     |        | 0  | 0  | 1  | 1  | 2  | 2  | 3  | 3  |
|             | write <span>D</span> for segment 2, clone... |        | 0  | -  | 1  | -  | 2  | -  | 3  | -  |
|             | write <span>S</span> for child...            |        | 8  | -  | 6  | -  | 7  | -  | -  | -  |





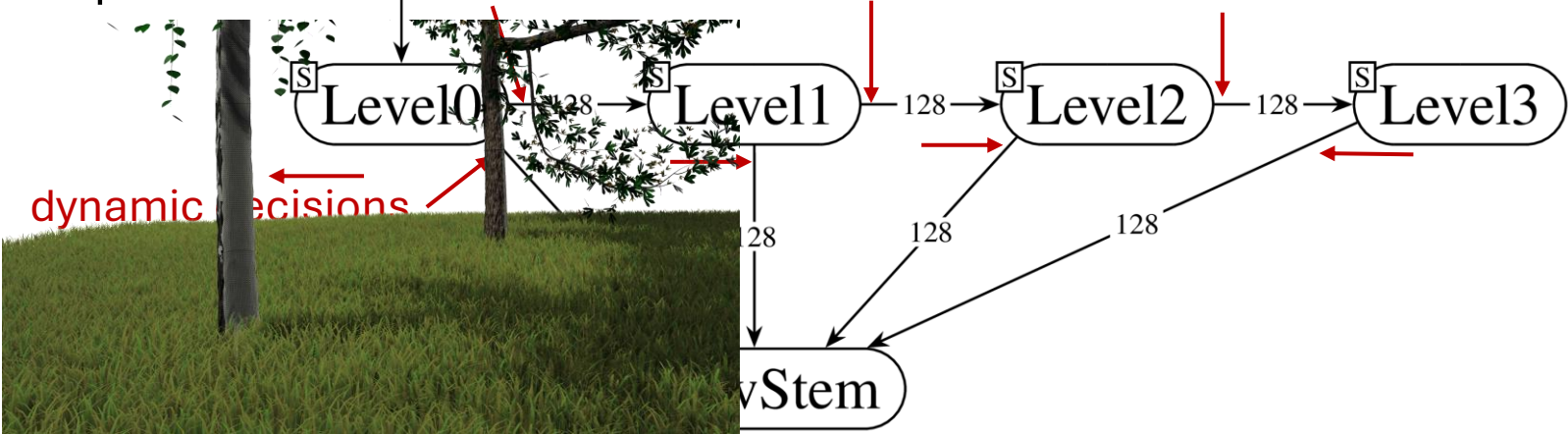
Generate tree parts  
that...

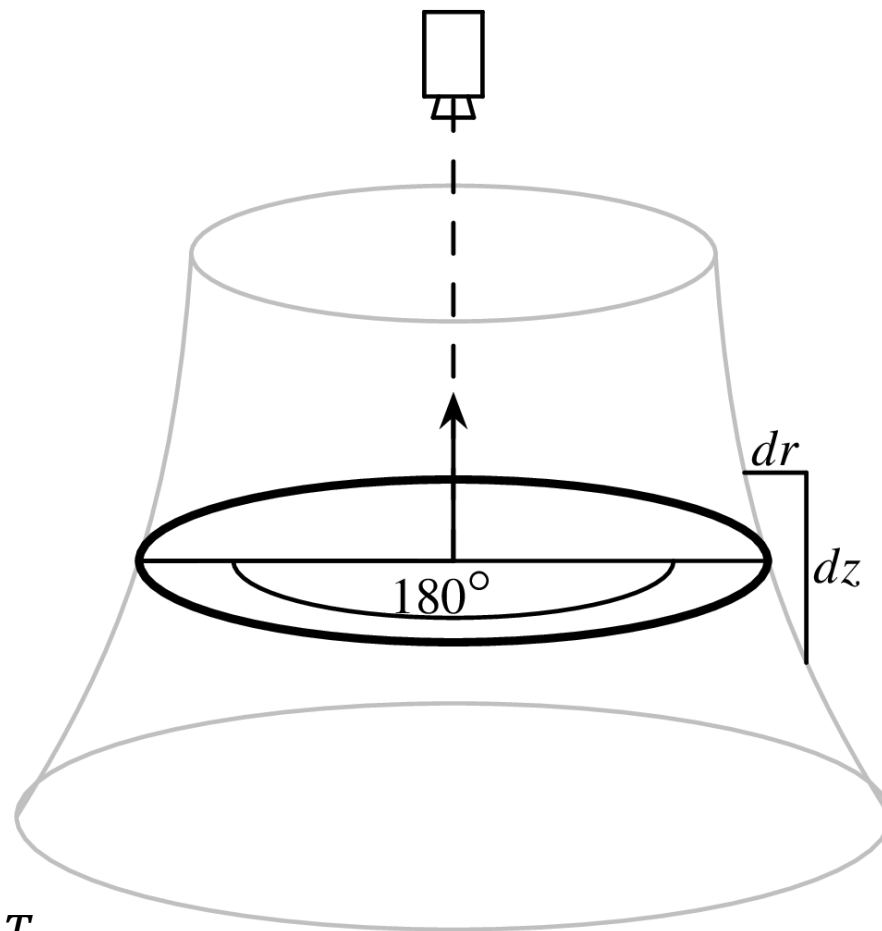
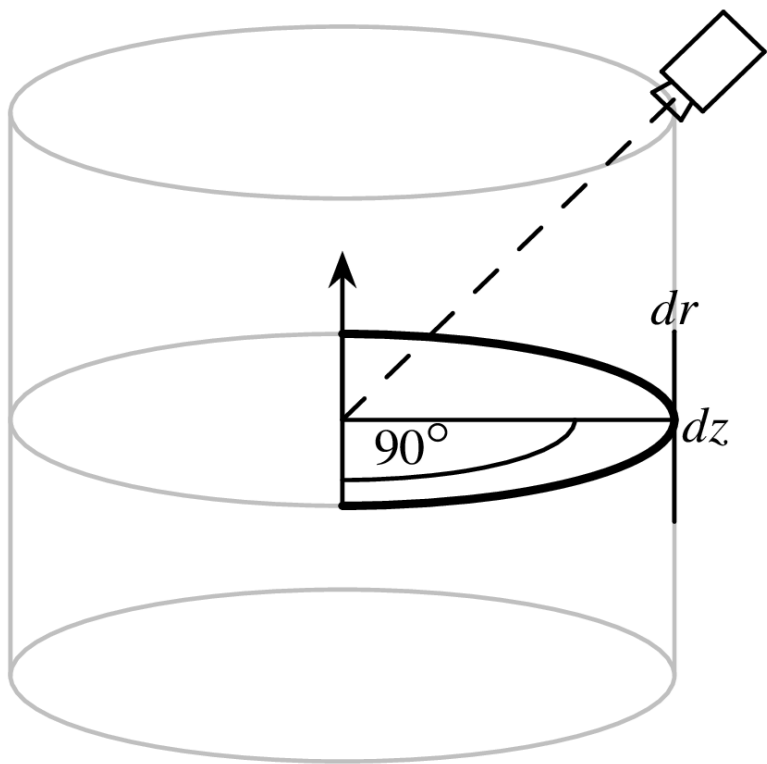


...are in the frustum...



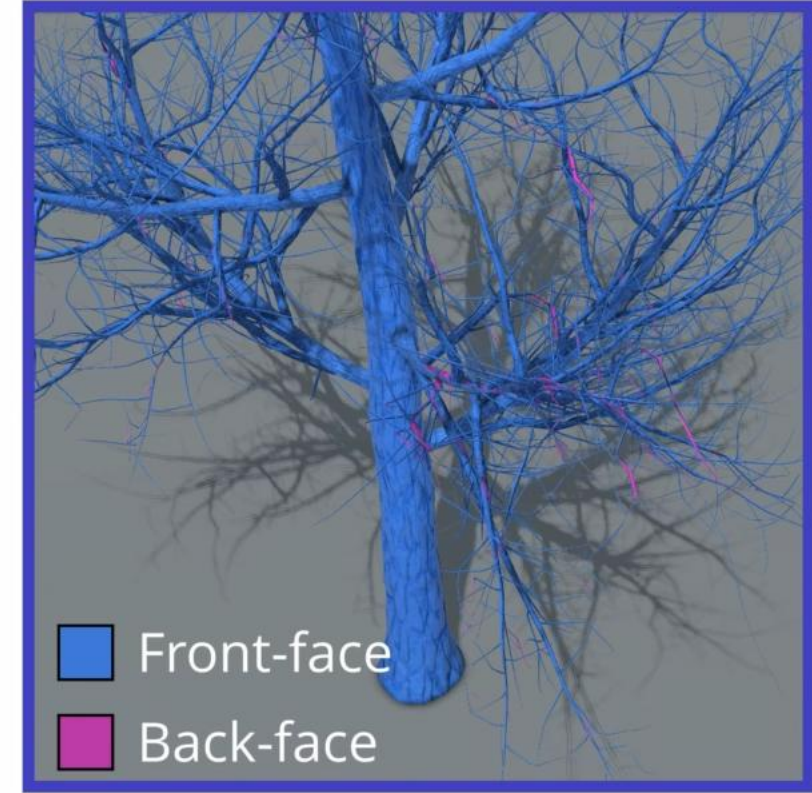
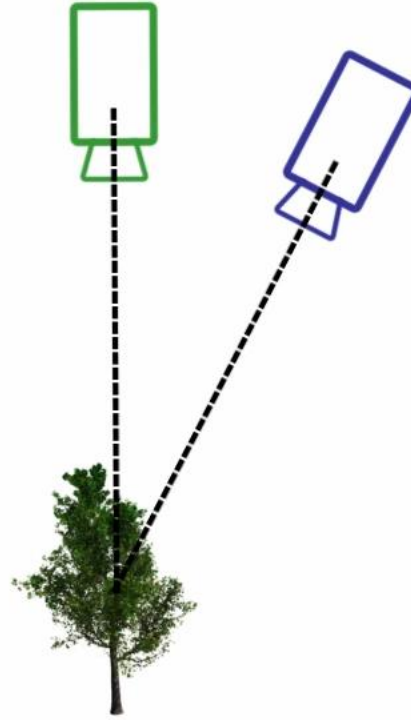
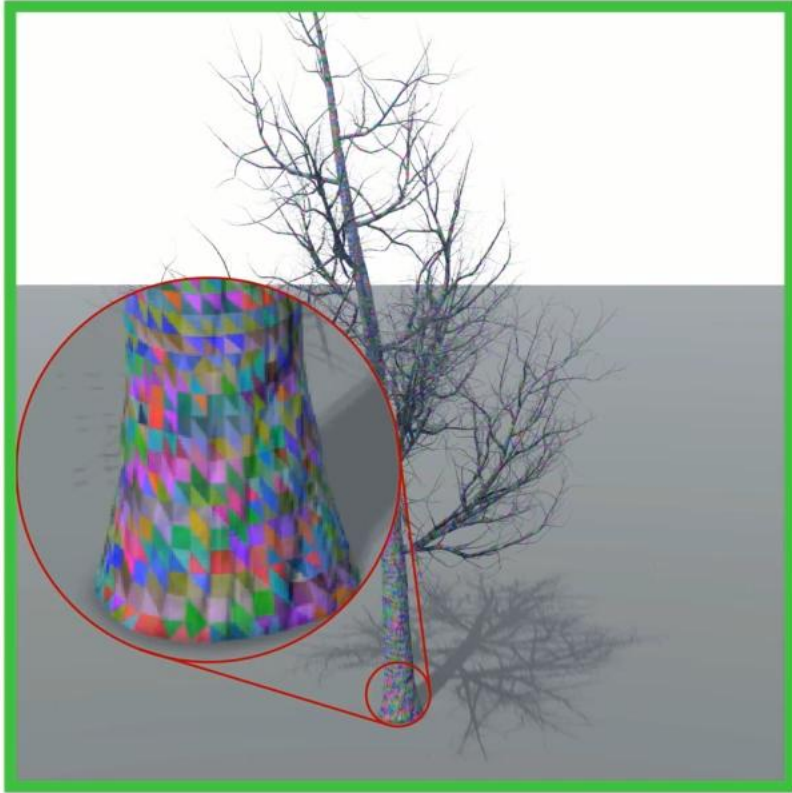
...point towards the camera...





$$\theta = \cos \left( \max \left( \frac{dr}{dz} \cdot \frac{x}{\sqrt{1-x^2}}, -1 \right) \right)$$

$z \cdot v^T$   

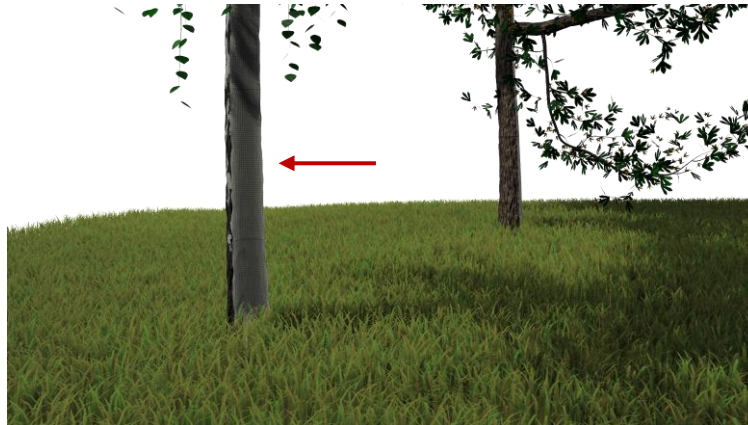
Generate tree parts  
that...



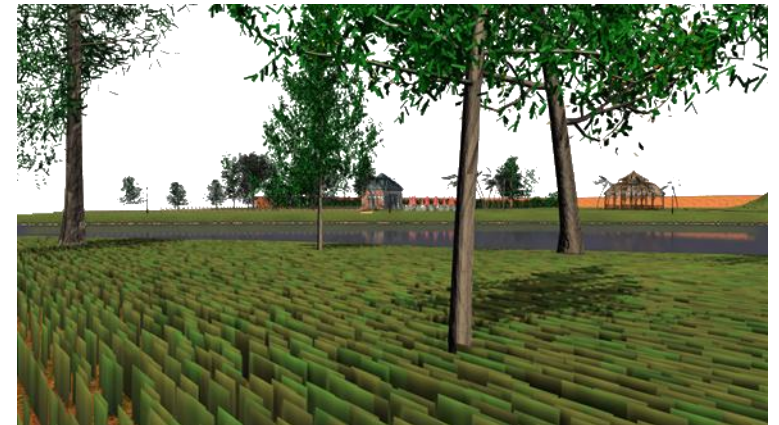
...are in the frustum...



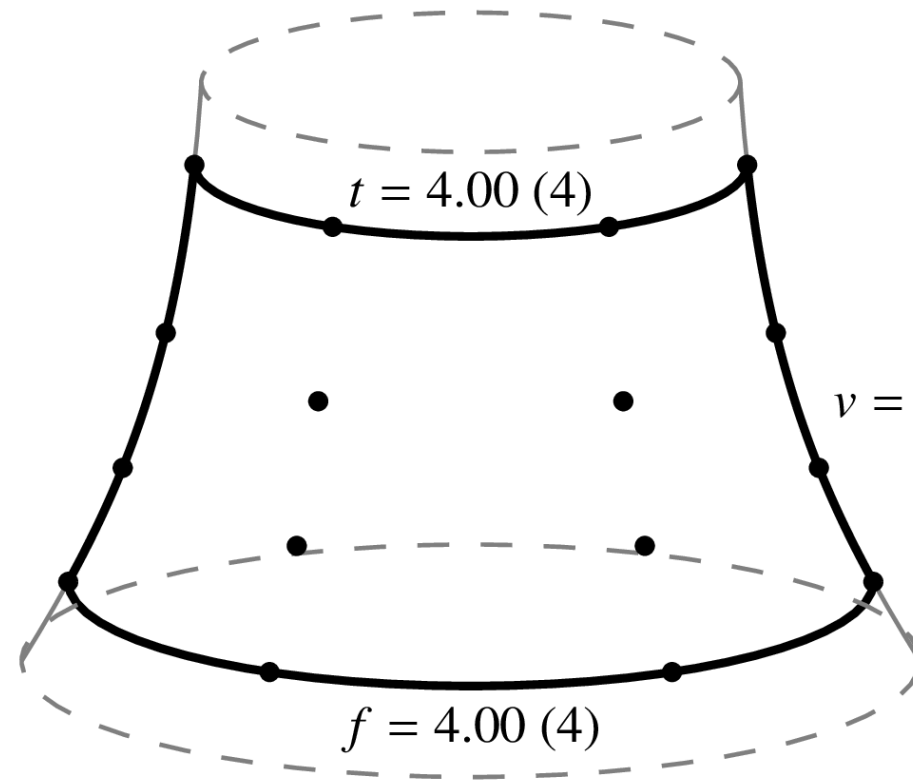
...point towards the camera...



...in the quality required.



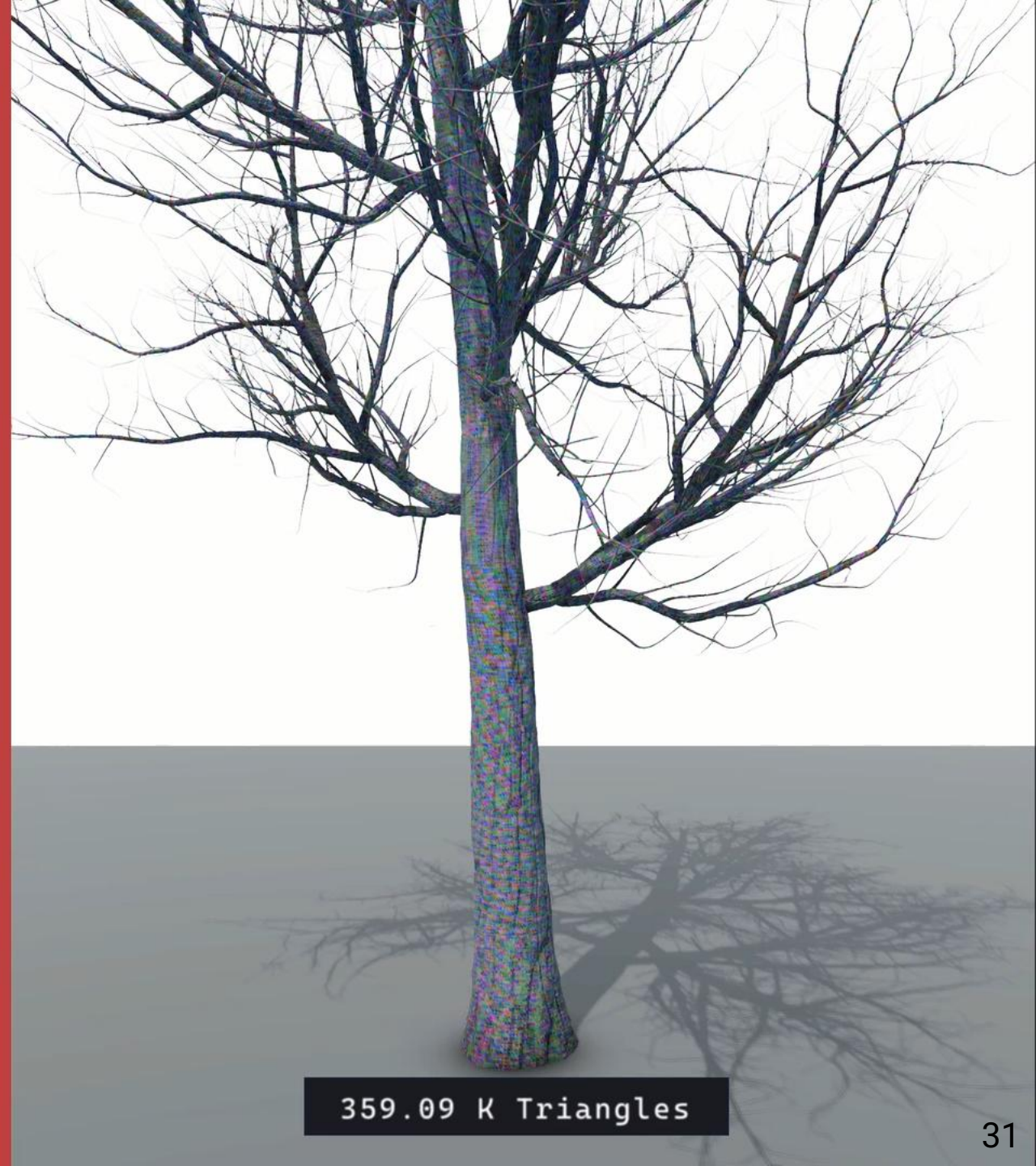
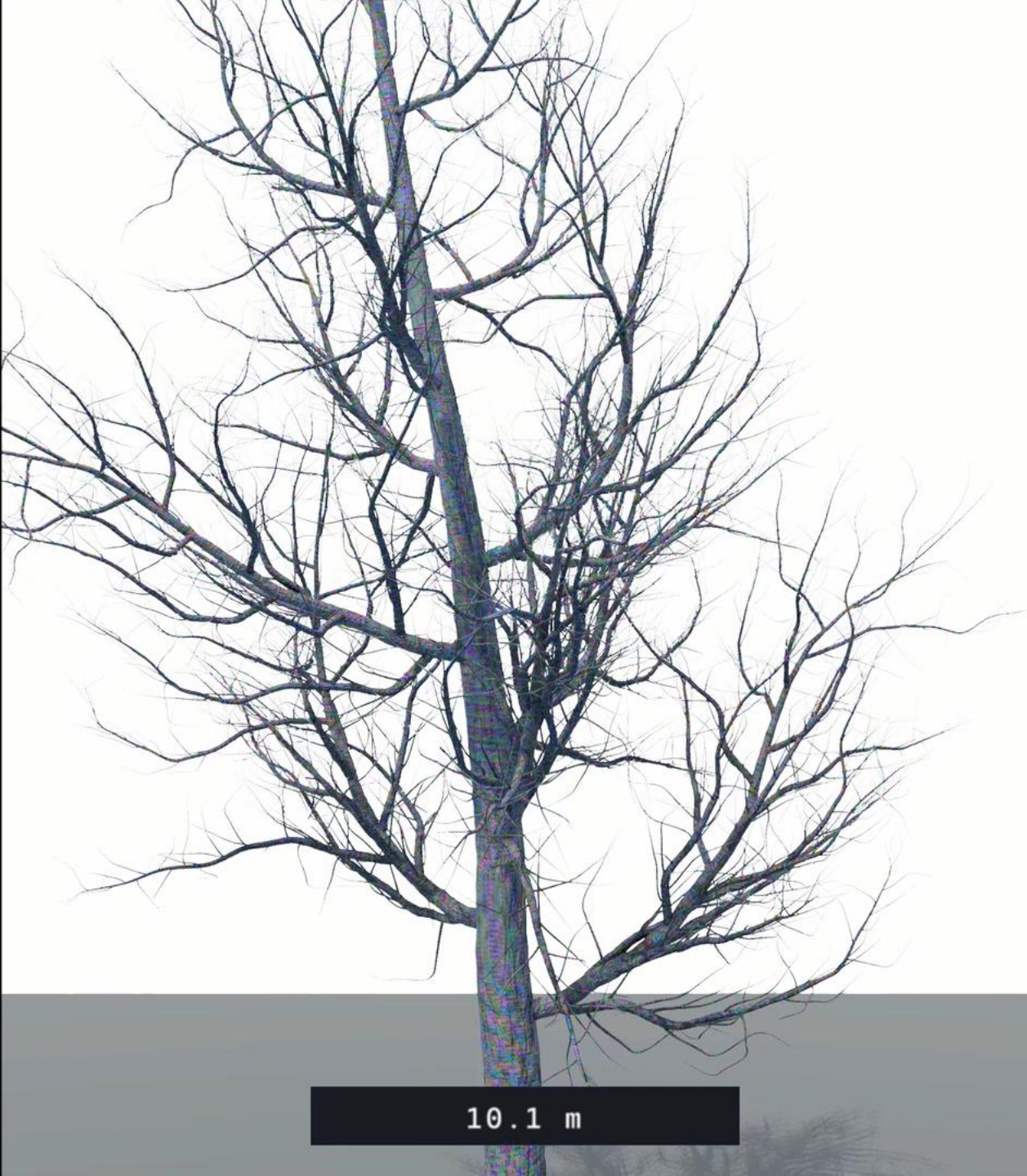
$$t = \frac{2 \cdot \theta_{\text{to}} \cdot r_{\text{to}}}{\Delta}$$

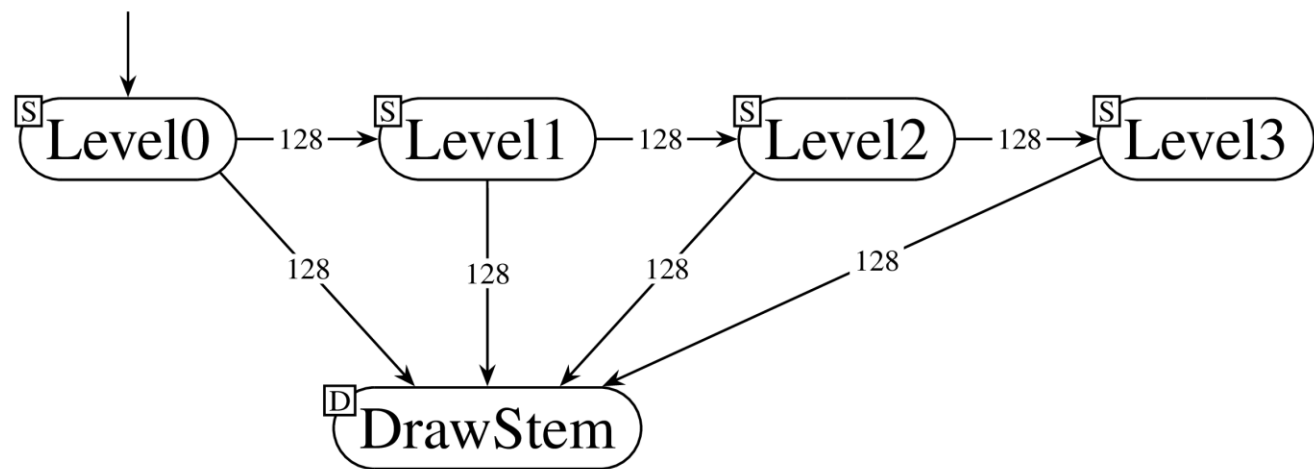


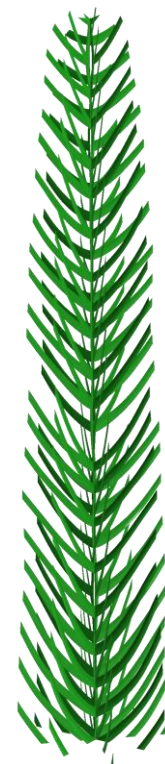
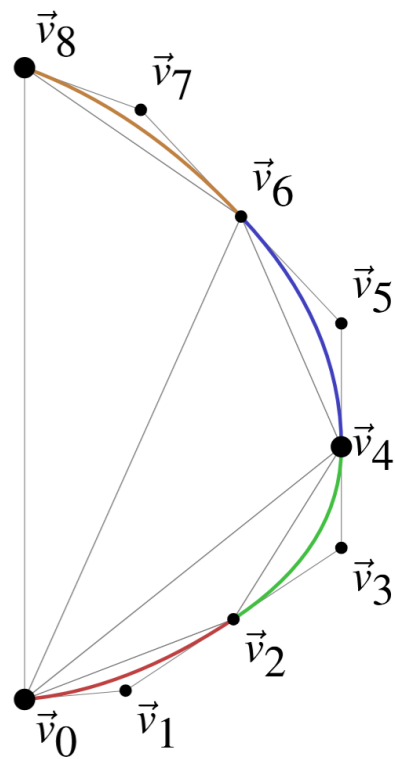
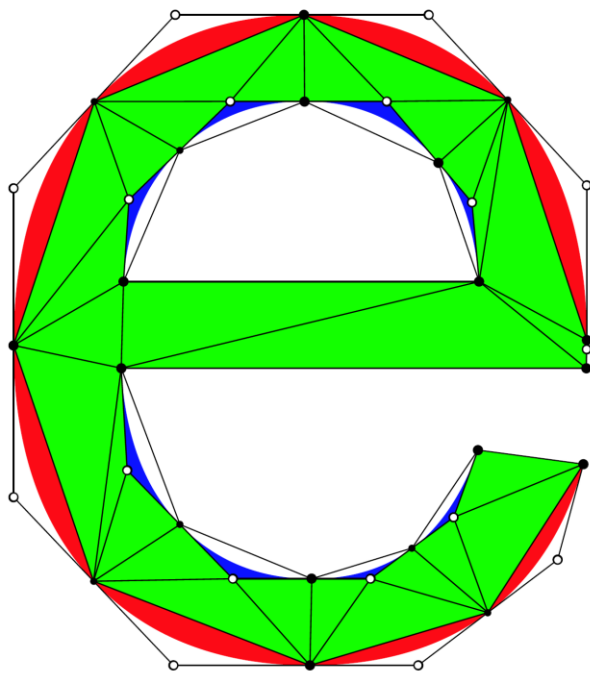
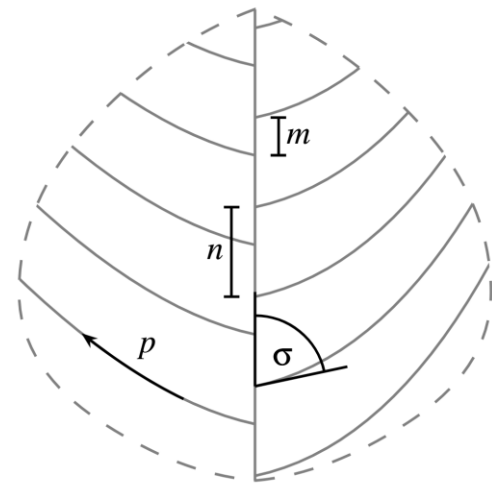
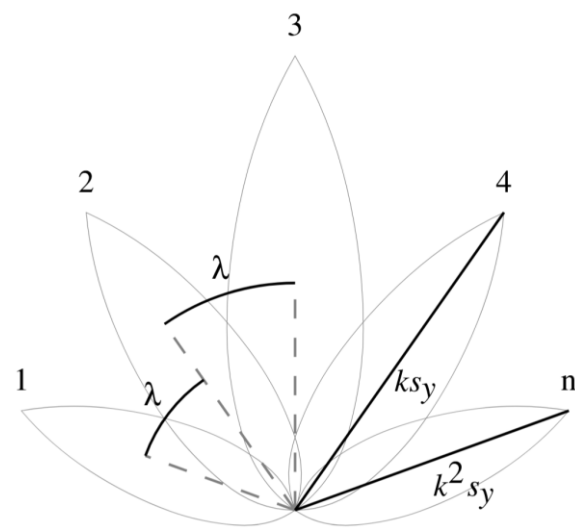
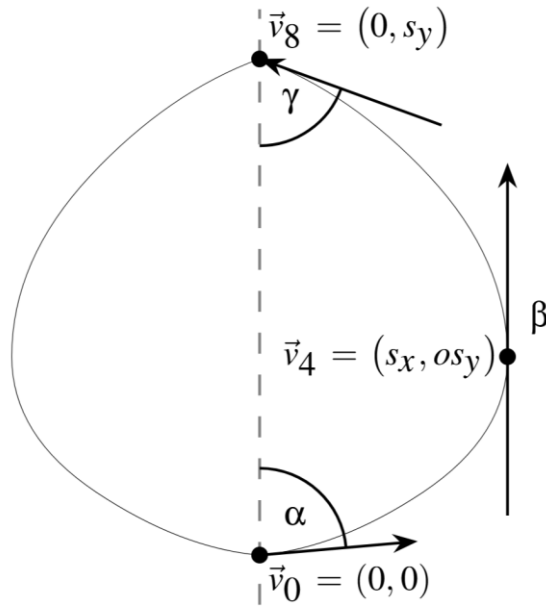
$$v = \frac{l}{\Delta}$$

$$f = \frac{2 \cdot \theta_{\text{from}} \cdot r_{\text{from}}}{\Delta}$$

sample distance

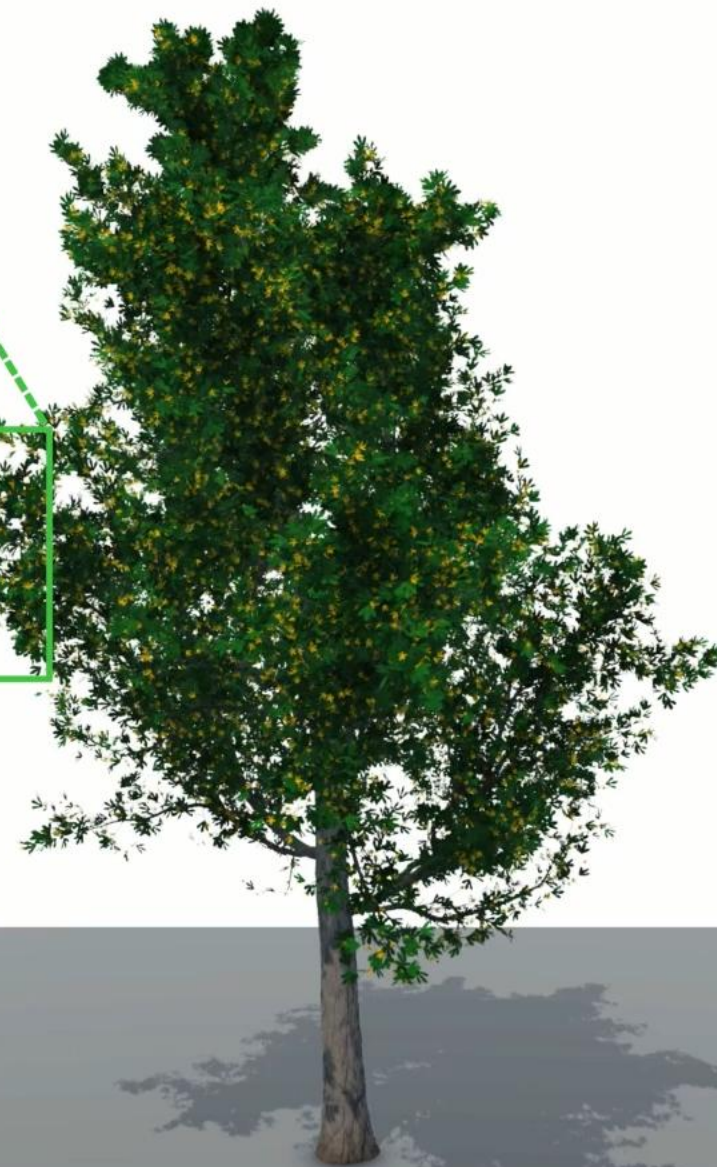






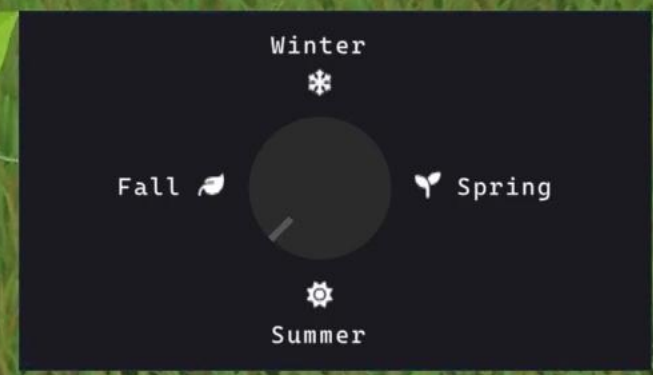


10.0 m



3.54 M Triangles







## Apple Tree Config

Branches

20

Branch Length

0.500

Attraction Up

0.000

Fruit Chance

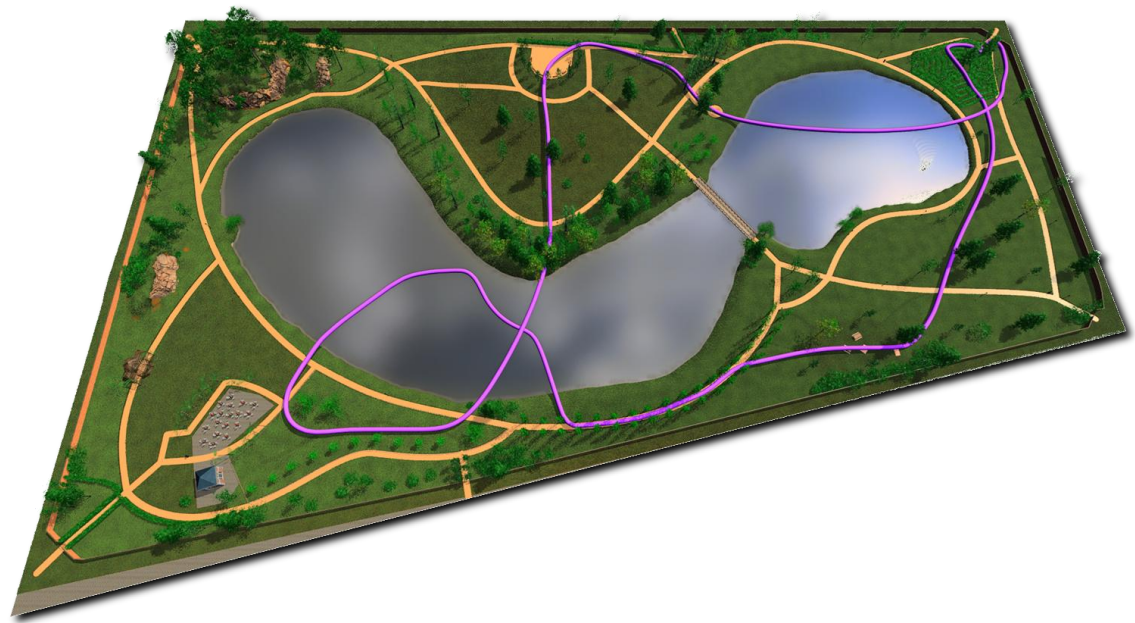
0.050

... over 150 more parameters!

Seed

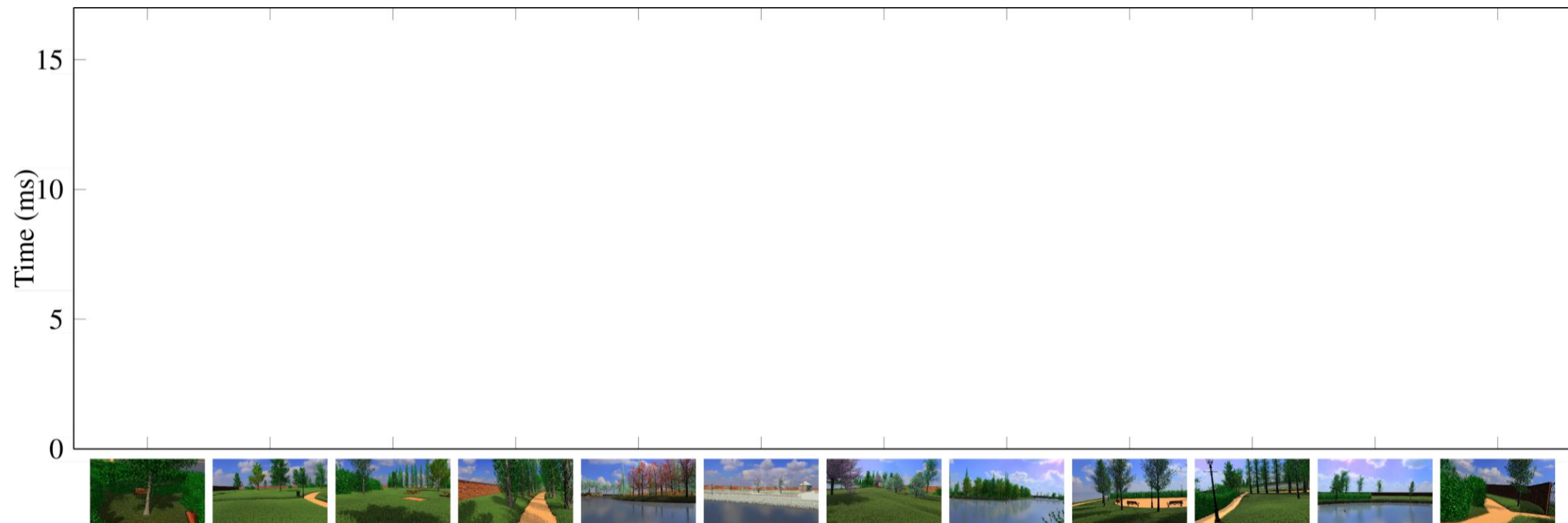
45

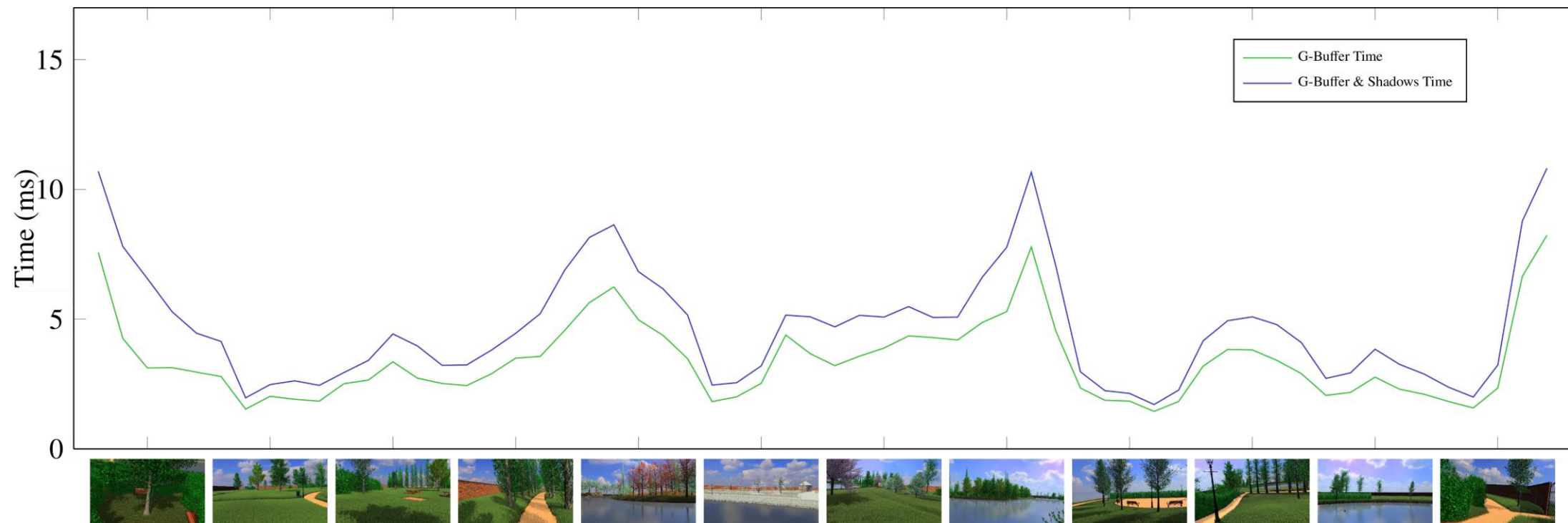


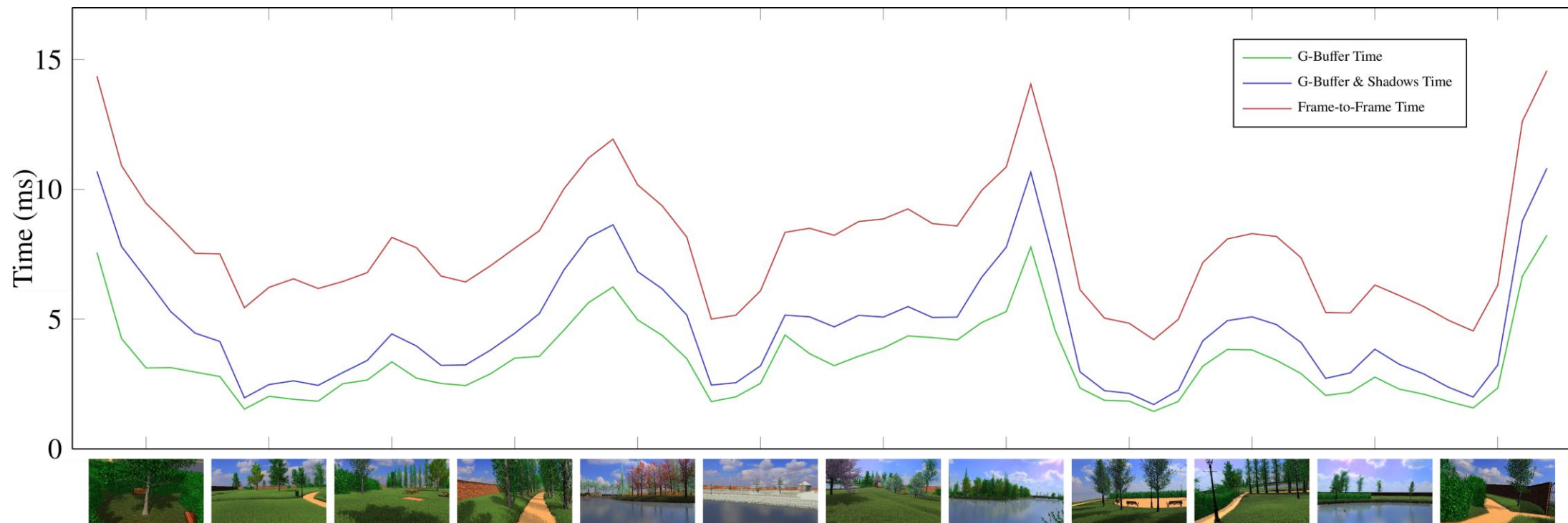


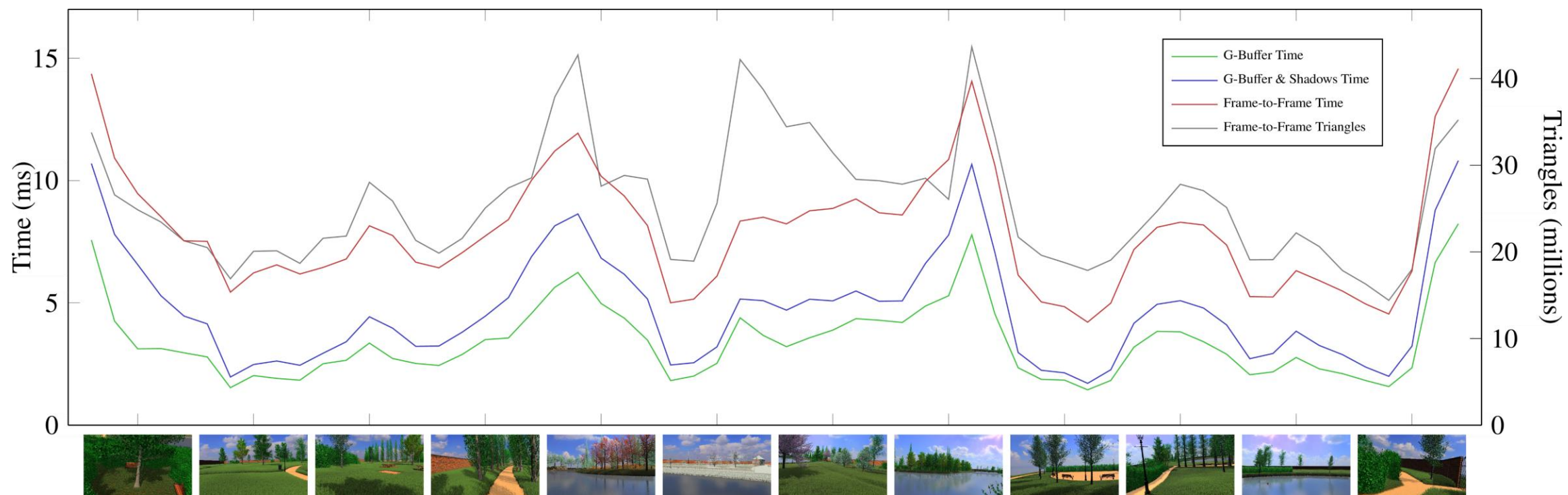


20 min  
1,4 km









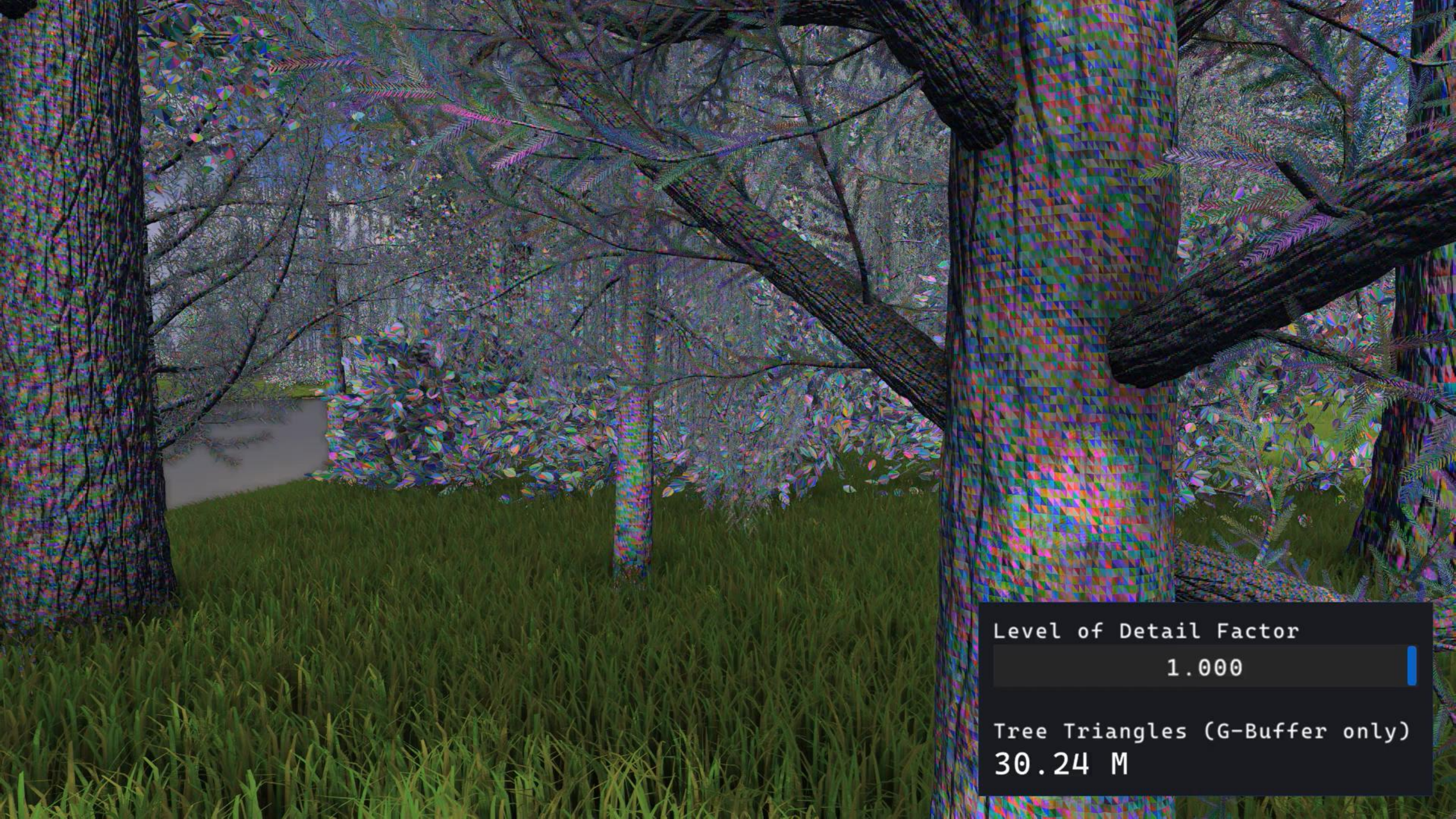


Level of Detail Factor

1.000

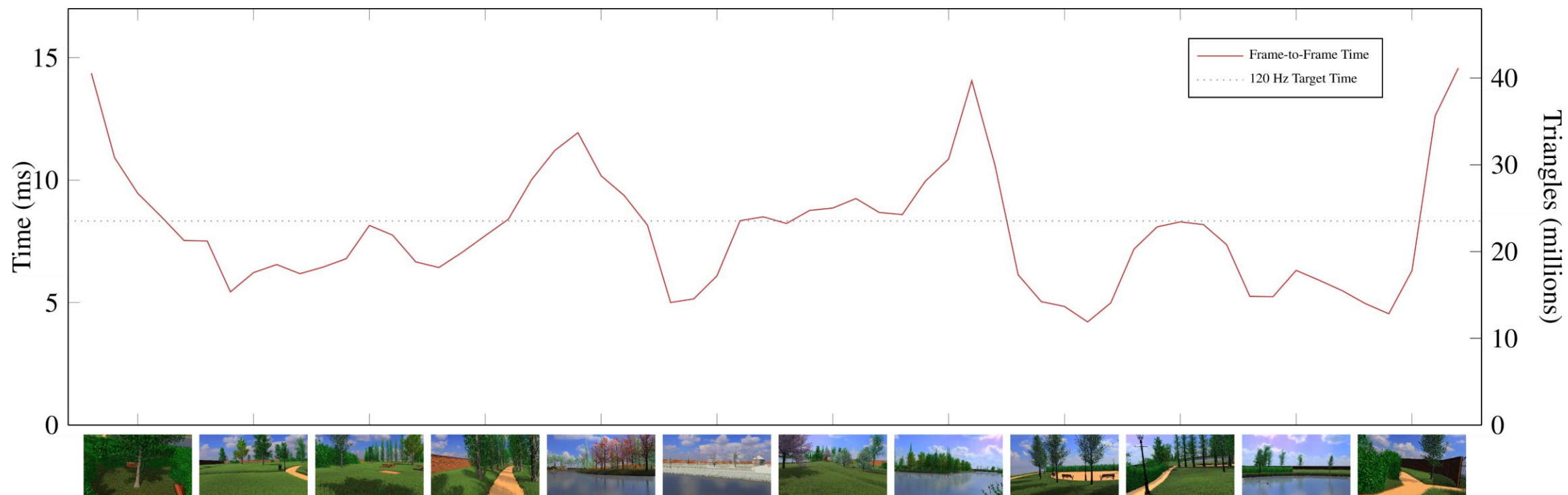
Tree Triangles (G-Buffer only)

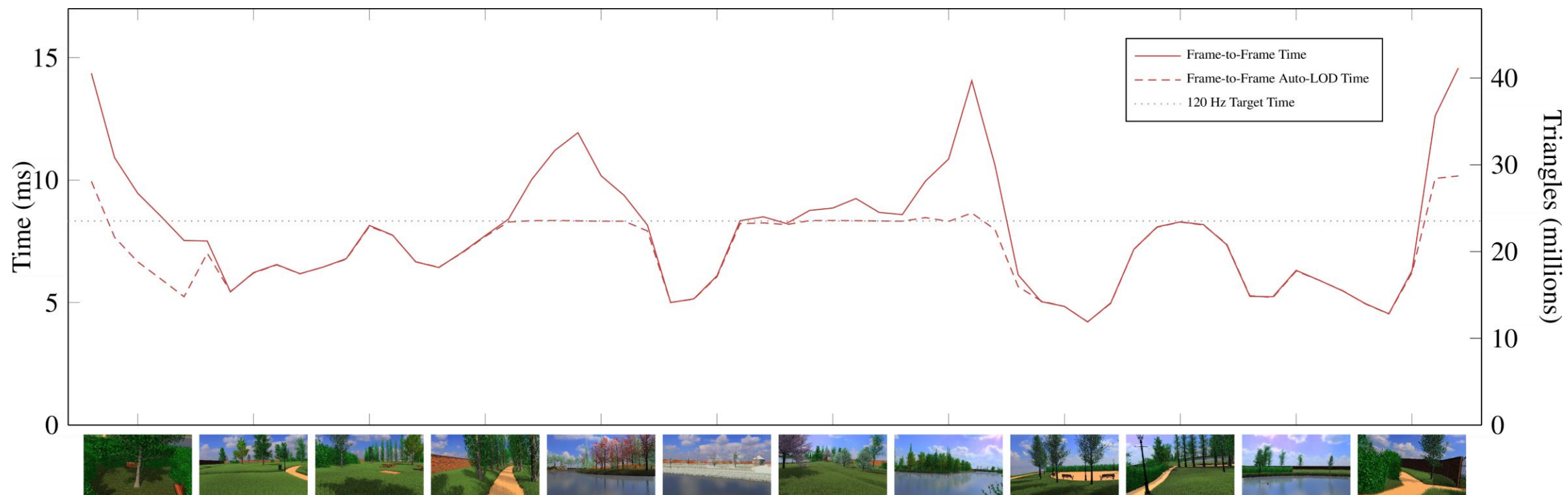
30.24 M



Level of Detail Factor  
1.000

Tree Triangles (G-Buffer only)  
30.24 M





```
struct Vertex {
    float3 position;
    float3 normal;
    float2 texCoord;
}; // 32 byte
```



|                       | One Sassafras Tree |          | Trees of Entire Scene |          |
|-----------------------|--------------------|----------|-----------------------|----------|
| <b>Stem Vertices</b>  | 1,479,896          | 45.2 MiB | 493,756,958           | 14.7 GiB |
| <b>Stem Triangles</b> | 2,038,580          | 23.3 MiB | 684,357,207           | 7.6 GiB  |
| <b>Leaf Vertices</b>  | 1,974,720          | 60.3 MiB | 302,936,912           | 9.0 GiB  |
| <b>Leaf Triangles</b> | 1,974,720          | 22.6 MiB | 302,936,912           | 3.4 GiB  |
| <b>Total</b>          | <b>151.4 MiB</b>   |          | <b>34.8 GiB</b> ⚡     |          |

→ Our system needs ~51 KiB



→ Culled Generation

→ Continuous LOD

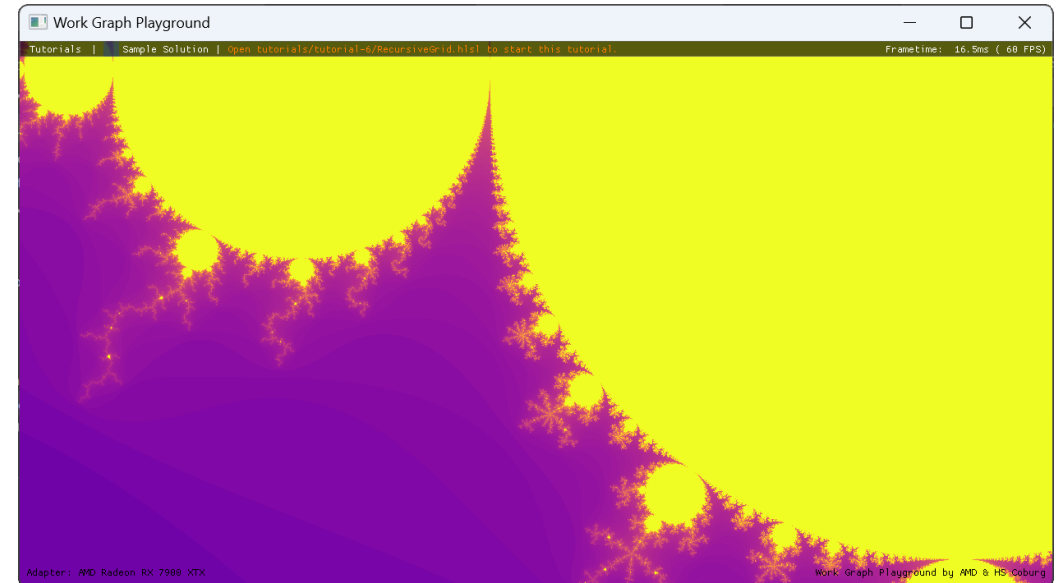
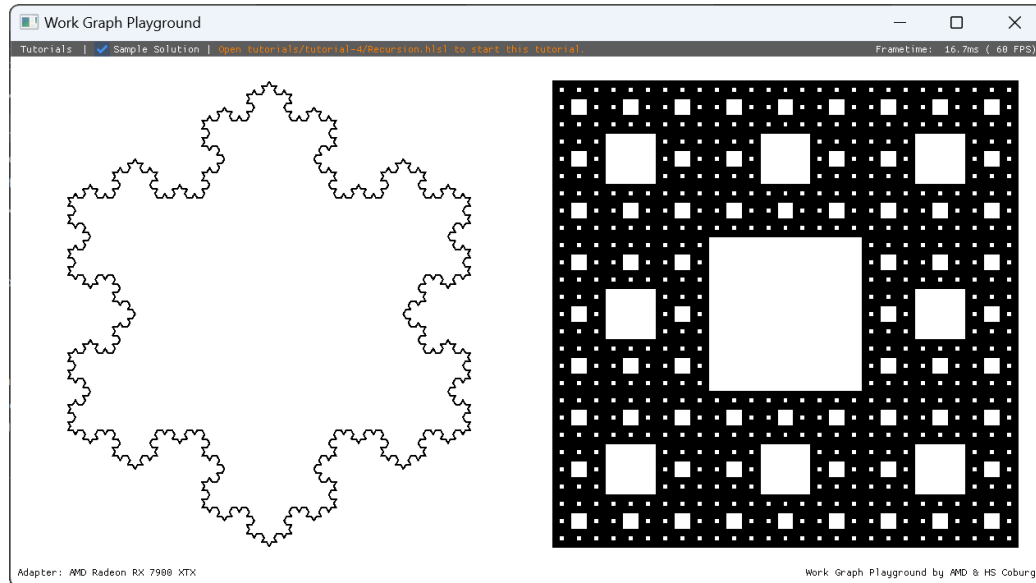
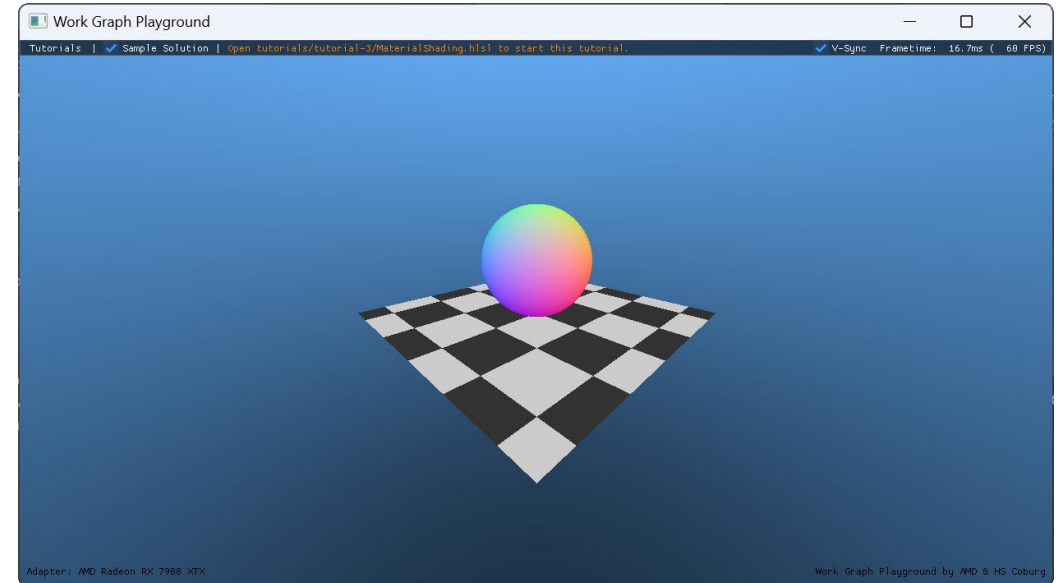
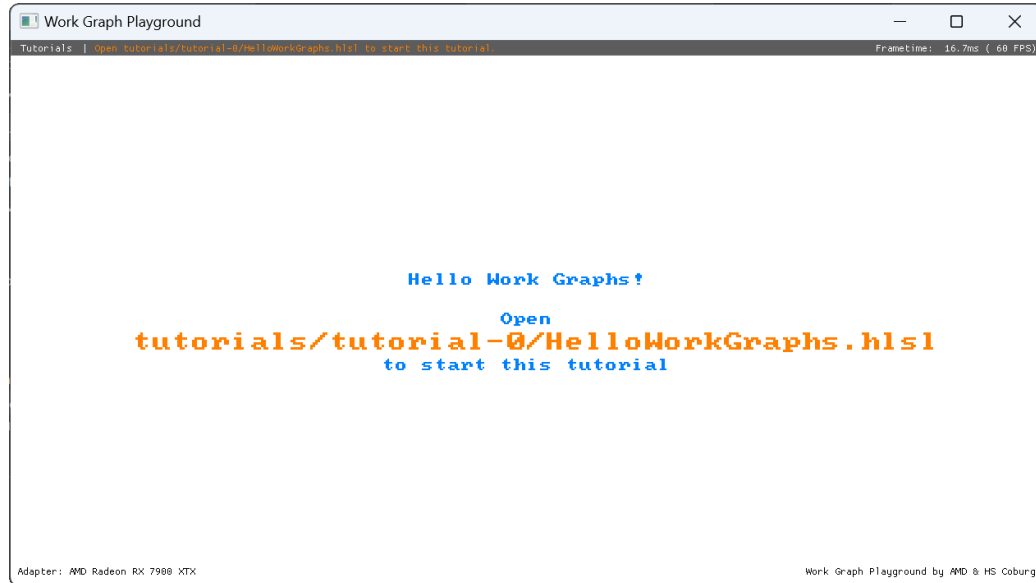
→ Seasons

→ Animation

→ Real-time Editing

→ Automatic LOD

# Work Graph Playground



# Tree Sample



<https://github.com/Bloodwyn/gptree>